

操作系统review

外设管理

- 源代码不用看
- 核心数据结构
- 磁盘高速缓存池
- 借助缓存的读写操作
- 缓存块互斥使用的过程
- 缓存队列的变化

Part4 外设和文件1 外设驱动

轮询 (polling)：是一种通过程序主动检查外设状态的方式。具体来说，操作系统定期（或不断地）检查外设是否准备好进行数据传输或处理请求。如果外设准备好（例如，输入设备接收到数据，或输出设备准备好接受数据），操作系统就会执行相应的操作。如果外设没有准备好，操作系统则继续检查，直到外设准备好为止。

效率比忙等高，但CPU仍要承担所有外设事务

2.2 中断驱动的IO方式

Unix 的 同步读，磁盘为例

IO请求管理

FIFS的硬盘
IO请求队列

磁盘中断处理程序（后台）

进程启动IO操作（前台，设备驱动）

1、现运行进程PA构造读IO请求块buf (mode: READ) 送IO请求队列尾部。原先队列空？
Y，根据buf中的info向外设发IO命令

2、Sleep(&buf,-50)

3、处理底层送来的外设新数据

1、IO出错否？Y，重发队首buf的IO请求（试10次）

2、in 128次磁盘数据端口，取数据缓存中的新数据，送buf指定的内存单元

3、WakeUpAll(&buf)

4、摘队首IO请求块

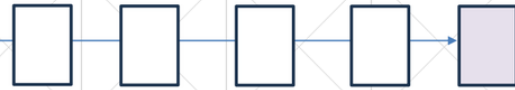
5、队列变空了吗？N，根据新队首buf中的info向外设发IO命令

中断驱动的IO方式

Unix 的 异步写，磁盘为例

IO请求管理

FIFS的硬盘
IO请求队列



磁盘中断处理程序（后台）

进程启动IO操作（前台，设备驱动）

产生需要写磁盘的数据块，首地址buffer

- 1、现运行进程PA构造写IO请求块buf (mode: WRITE) 送IO请求队列尾部。
原先队列空? Y, 根据buf中的info: out 128次磁盘数据端口, 将buffer中的数据写入磁盘数据缓存。向外设发写命令

- 1、IO出错否? Y, 重发队首buf的IO请求 (试10次)

- 2、in 128次磁盘数据端口, 将buffer中的数据写入磁盘数据缓存, 向外设发写命令

- 3、WakeUpAll(&buf)

- 2、摘队首IO请求块

- 3、队列变空了吗? N, 根据新队首buf中的info向外设发IO命令

中断驱动的IO方式:

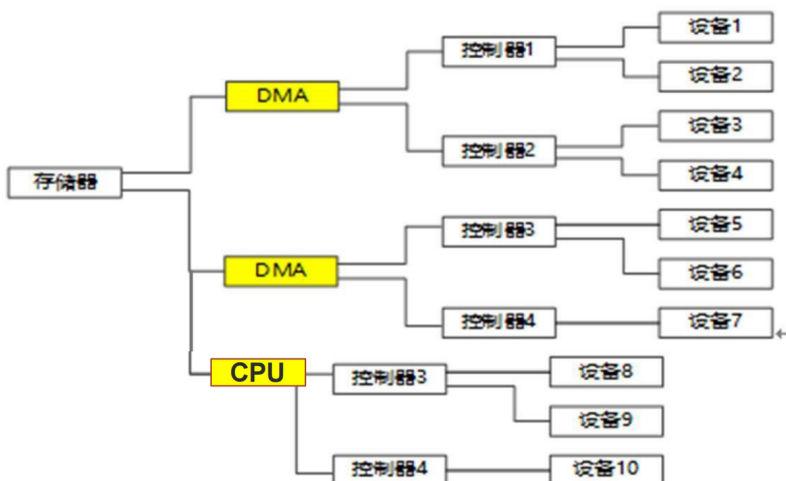
- 中断是IO完成信号。CPU无需忙等或轮询 IO操作。 计算和IO可以并行。不同的外设, IO也可以并行。
- 数据块从外设数据缓存复制到内存的工作由CPU完成。所以, 中断驱动的IO方式只适合键盘、鼠标 等数据缓存很小的外设

DMA 直接内存访问方式:

数据就绪于外设数据缓存后, 外设控制器向DMA控制器发中断信号。

DMA控制器循环, 盗取主存周期, 把数据缓存中的数据送入内存, 一次传送一个字。这一步会循环直至内存得到数据缓存中的所有数据。

所有数据传送完毕后, DMA控制器向CPU发中断信号。中断处理程序不复制数据。CPU完全不参与数据传输。



Part4 外设和文件2 缓存

内存中为每个外设配缓存，暂存外设IO数据。

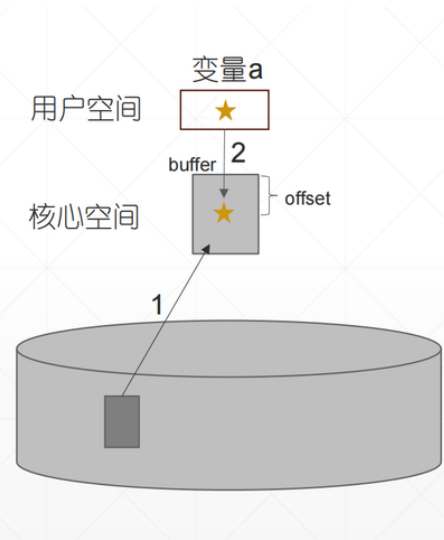
缓存在核心空间。Unix V6++，在内核数据段。

TTY类：主要负责终端设备的管理和与用户的交互，处理终端的输入输出流。

TTY_Queue类：是 TTY 类的一部分，专注于管理输入输出的数据队列，用于缓存和流控制，确保数据有序、及时地传输。

每个缓存块同步一个磁盘数据块。一个磁盘数据块占据某张磁盘连续的多个扇区。

写操作（变量和数组）：



内核，应用程序变量a写盘（写入数据量不足一个数据块，先读后写）

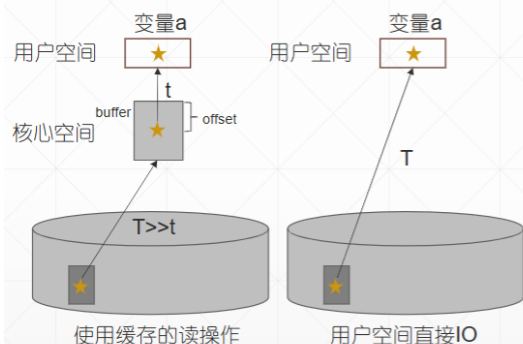
- 计算数据data所在的磁盘数据块号blkno，找到分配给blkno的缓存块buffer
- (1) if(buffer尚未与blkno同步，数据不可用)
启动IO操作读入磁盘数据块，入睡等待IO完成
- 计算data在缓存块中的偏移量offset
- (2) 将变量a复制到缓存块中（送数据至缓存，数据传输）
`buffer[offset] = a;`
- 结束

内核，应用程序变量a写盘（写入一整块，无需先读）

- 计算数据data所在的磁盘数据块号blkno，找到分配给blkno的缓存块buffer
- 将用户空间数据块写进缓存块
`buffer = 数组a;`
- 结束

缓存的价值

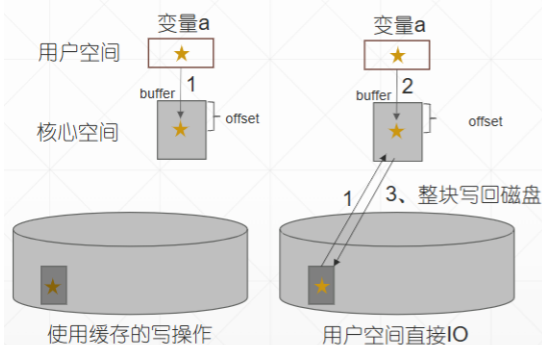
性能 for 读操作



计算读操作耗时

- 缓存命中（缓存块中有可用数据）， t
- 缓存不命中， $T + t$
- 若缓存命中率为 95%，读磁盘
平均耗时 = $(T + t) * 5\% + t * 95\%$
 $= T * 5\% + t \ll T$ (用户空间直接IO)
- 缓存的价值 (1) 数据重用
(2) 减少读操作数量，提高IO操作的效率

性能 for 写操作

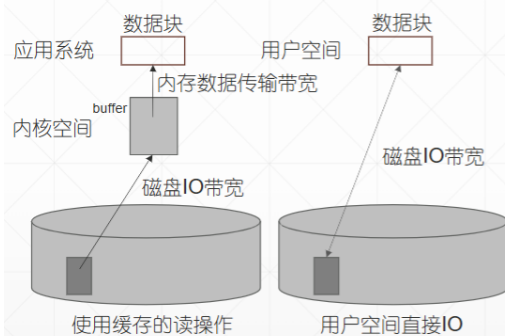


写操作耗时

- 若缓存命中， t 。缓存不命中， $2T + t$ 。
- 若缓存命中率为 95%，写磁盘
平均耗时 = $(2T + t) * 5\% + t * 95\%$
 $= T * 10\% + t \ll 2T + t$ (用户空间直接IO)
- 缓存的价值 (1) 缩短写操作耗时
(2) 减少写操作数量，提高IO操作的效率
- 代价：丢数据，降低文件系统一致性。

不用缓存的情况：大数据批处理任务

用户空间IO的使用场合，其一



大数据批处理任务

批处理（遍历）G、T 规模的大数据集
(应用的特征：用一次，数据没有重用可能)

关闭缓存层，消除大数据集在内存中的复制开销。

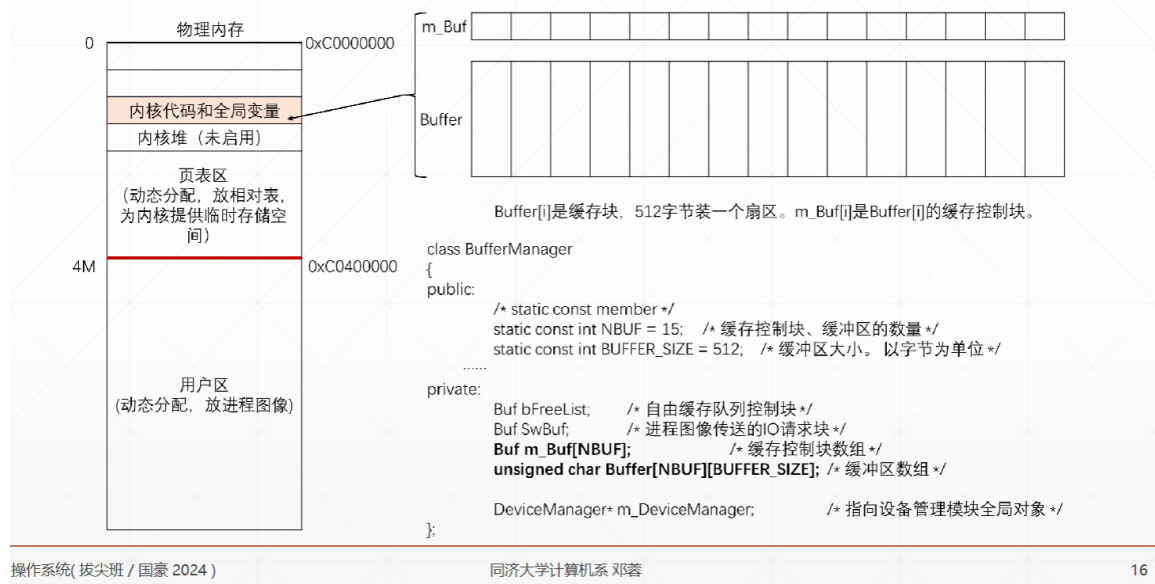
数据集只读、不可更改。数据若有更新，将新数据写入新分配的数据块，甚至是新文件。

磁盘高速缓存池

4、磁盘高速缓存

- 在通用计算机系统中，内核设置磁盘高速缓存池，暂存部分磁盘数据块。
- 磁盘高速缓存池，所有块设备共享。
- 每个缓存块可以存放一个磁盘数据块，这块数据供整个系统共享。需要的进程互斥使用。配一个缓存控制块，登记这个缓存块的使用情况。

5、Unix V6++的磁盘高速缓存池



LRU的自由缓存队列



- 缓存块互斥使用。
- 自由缓存队列**, 管理B_BUSY标识为0的缓存块。这些缓存块随时可用。
- 缓存块使用前上锁 (B_BUSY=1)、从自由缓存队列移除。用完解锁 (B_BUSY=0), 送队列尾部。
- 这是一条LRU (Least Recently Used) 队列, 队首是最近最久未访问数据块。
- 系统需要使用的数据块缓存不命中时, 分配给它自由缓存队列队首缓存块, 原有数据块被淘汰。

读是同步请求, 影响IO系统响应速度, 优先级高。

- 读操作通常是**同步请求**。当进程发起一个读操作时, 它会**暂停执行后续的指令**, 等待读操作完成, 即数据从磁盘读取到内存 (或缓存) 并可供使用后, 进程才会继续执行。
- 当读操作完成后, 数据已经准备好, 此时就需要唤醒等待这个数据的进程, 让它能够继续执行后续的操作。这是因为进程在等待读操作完成期间是处于阻塞状态的, 只有读操作完成后将其唤醒, 它才能继续工作。

写操作是异步的, 优先级低。

- 写操作是**异步的**, 当进程发起写操作后, 它可以继续执行其他任务, 而**不需要等待写操作实际完成** (即将数据从内存写入磁盘)。

- 由于进程没有因为写操作而进入等待状态，所以当写操作完成时，通常不需要唤醒进程。不过在某些特殊情况下，比如进程在执行某些操作时需要确认数据已经成功写入磁盘，这时可能需要对进程进行唤醒操作，但这种情况相对较少，并且不属于写操作的一般处理流程。

4、磁盘数据块为什么要先读后写

磁盘写操作以数据块为单位，会用内存中的整块数据覆盖磁盘数据块。所以，正确的操作步骤是（1）先读，确保缓存块中包含磁盘数据块中的旧数据（2）后写，将新数据存入缓存块（3）硬件（DMA 控制器和磁盘控制器）将既包含新数据，又包含旧数据的缓存块写回磁盘。如果没有先读操作，内存中的错误信息会覆盖磁盘数据块中的部分数据，导致文件内容出错。

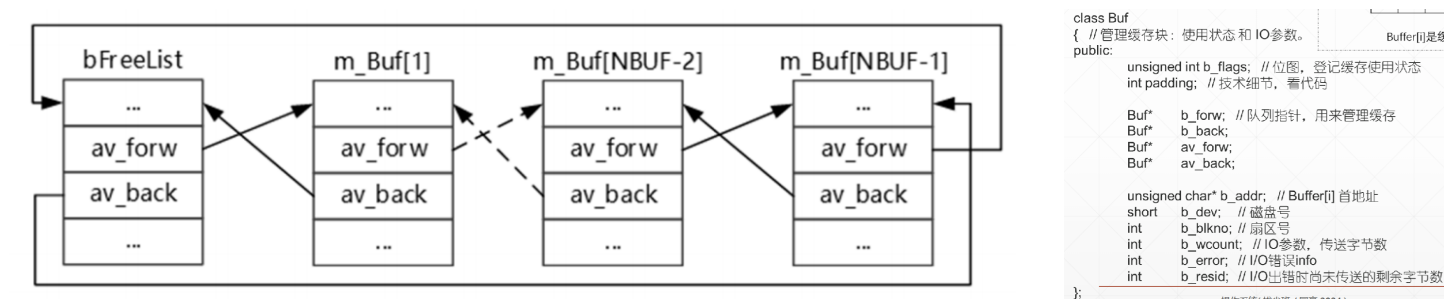
Unix V6写操作执行时机：

- 脏缓存块分配用来装新数据块时写回磁盘。
- 每30s执行一次update程序，将所有脏缓存块写回磁盘。

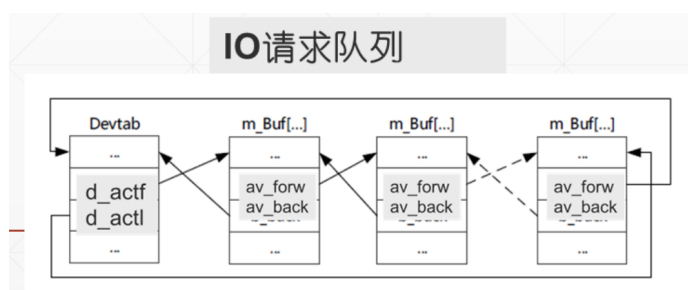
Unix V6++：每张磁盘dev有自己的设备缓存队列，管理该磁盘的数据块。

每张磁盘有自己私有的设备缓存队列和IO请求队列

自由缓存队列：



设备缓存队列

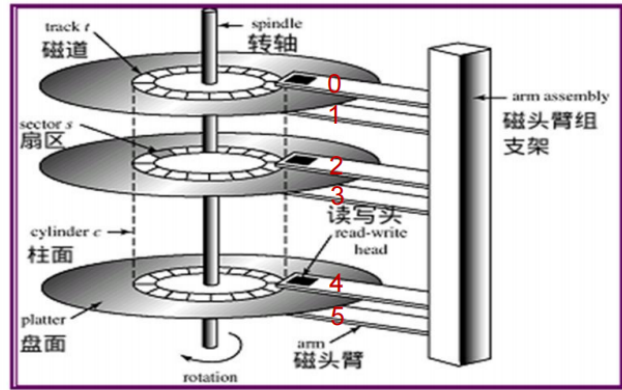


IO请求队列

```
class Devtab // 块设备表
{
    public:
        int d_active; // 磁盘忙，IO请求队列非空
        int d_errcnt; // 队首IO操作，出错次数
        Buf* b_forw; // 设备缓存队列队首
        Buf* b_back;
        Buf* d_actf; // IO请求队列队首
        Buf* d_actl;
};
```

磁道

- 机械硬盘，同心圆盘。
 - 每个面，布有用来存储信息的许多同心磁道。从外至内编号，0, 1, 2, 3。。。磁道号。所有盘面，相同磁道号的磁道组成柱面。柱面号就是磁道号。
 - 每个磁道，许多等圆心角圆弧。每个圆弧1个扇区。0, 1, 2, 3。。。扇区号。
 - 每个圆盘，上下2个盘面，每个盘面配有一个用来读写的磁头。磁头编号，0, 1, 2, 3。。。磁头号。
- 磁盘，信息存储、访问单位为扇区，目前的技术标准是每个扇区存放512字节。扇区的物理地址：柱面号、磁头号、扇区号。序列化，得扇区的逻辑地址。



0#柱面, 0#磁头: 逻辑扇区号 0~499
0#柱面, 1#磁头: 逻辑扇区号 500~999
1#柱面, 0#磁头: 逻辑扇区号 1000~1499
1#柱面, 1#磁头:

- **Seek time**寻道时间：磁头从当前柱面移动到目标柱面的时间。平均寻道时间小于10ms。
- **Rotational delay**旋转延时：磁头到达目标柱面（磁道）后，等待欲访问扇区转到磁头之下。平均旋转延时为磁盘旋转一周所需时间 T 的 $1/2$ 。
- **Data transfer**数据传输时间：磁头走过目标扇区，期间读扇区内容至硬盘数据缓存。若每个磁道包含 N 个扇区。数据传输时间为： T/N 。

FIFS (FIFO)：先来先服务

SSTF (Shortest Service Time First) :优先响应离磁头当前位置近的请求

SCAN (电梯调度算法)：考虑磁头的当前位置和运动方向，依次服务途径的所有磁道上的服务请求。当磁头到达最高或最低磁道时，改变运动方向。

C-SCAN：与SCAN类似，磁头始终一个方向运动

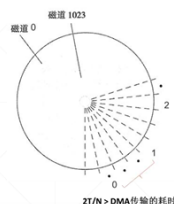
N-Step-SCAN和FCSCAN

- N 个IO请求队列。
 - 磁盘服务其中的一个队列。服务期间，新IO请求放在第一个可供使用的新队列。
 - 磁盘FIFS地服务所有队列。
 - 当前服务队列空，服务下一个队列。
 - 新请求队列满了，插入下一个队列。
 - 所有队列，轮流使用。
- 队列1 ** ← 当前服务队列
队列2 *****
队列3 *****
队列4 *****
队列5 *** ← 新请求队列
队列6
- 单个队列服务期间，可以使用任意调度算法。如果是SCAN，这个算法就叫做 N-Step-SCAN。
 - FCSCAN， $N=2$ 的特例，2个队列：磁臂单向移动，IO期间，新请求插入新队列。

例

移动硬盘，技术参数如下：转速7500rpm，平均寻道时间为4ms，每磁道500扇区，同磁道内相邻逻辑扇区错开2个物理扇区。问：读取一个1.28M（十进制）字节的文件需要多长时间？

- 每分钟7500转，旋转1周时间为 8ms。
- 平均的旋转延迟 4ms
- 一个扇区的读取时间为 8ms/500
- 1.28M=2500 sector（扇区）



若2500扇区完全随机存放。

访问每个扇区都要经历寻道和旋转延时：

$$4\text{ms} + 4\text{ms} + (8\text{ms}/500)$$

顺序读取该文件需要：

$$2500 * (4\text{ms} + 4\text{ms} + (8\text{ms}/500)) = 20.04\text{s}$$

若2500扇区顺序排在相邻5个磁道内。假设，文件的0#数据块放在第一个磁道的0#扇区。所有磁道在同一个柱面。

访问第一个500个扇区：

$$\text{耗时} \approx 4\text{ms} + 4\text{ms} + 8\text{ms} * 3 = 32\text{ms}$$

后续4个磁道不需要寻道，每个磁道的访问时间：

$$\text{耗时} \approx 8\text{ms} + 8\text{ms} * 3 = 32\text{ms}$$

顺序读取该文件需要耗时 $\approx 32 + 32 * 4 = 160\text{ms}$

相邻扇区号对应的扇区，物理上一定不能相邻（DMA数据传输期间，磁头一定会**刚好**错过下个物理相邻的扇区首部）。
∴在同一磁道上，**n扇区和n+1扇区之间一定会相隔若干个扇区**。
下面的问题是：应该相隔多少个物理扇区为好呢？
答案是：磁头越过这几个扇区需要的时间必须大于DMA芯片将512字节从磁盘数据缓存读入内存所需的时间。
精确的值需要根据硬件参数仔细计算，并留一定余量。

预读技术：

Unix V6，顺序读文件时进行预读：访问文件当前数据块时，顺带将下一个数据块读入缓存。

现代操作系统，一次性把包含当前数据块的8个文件块读入缓存。

PRD表

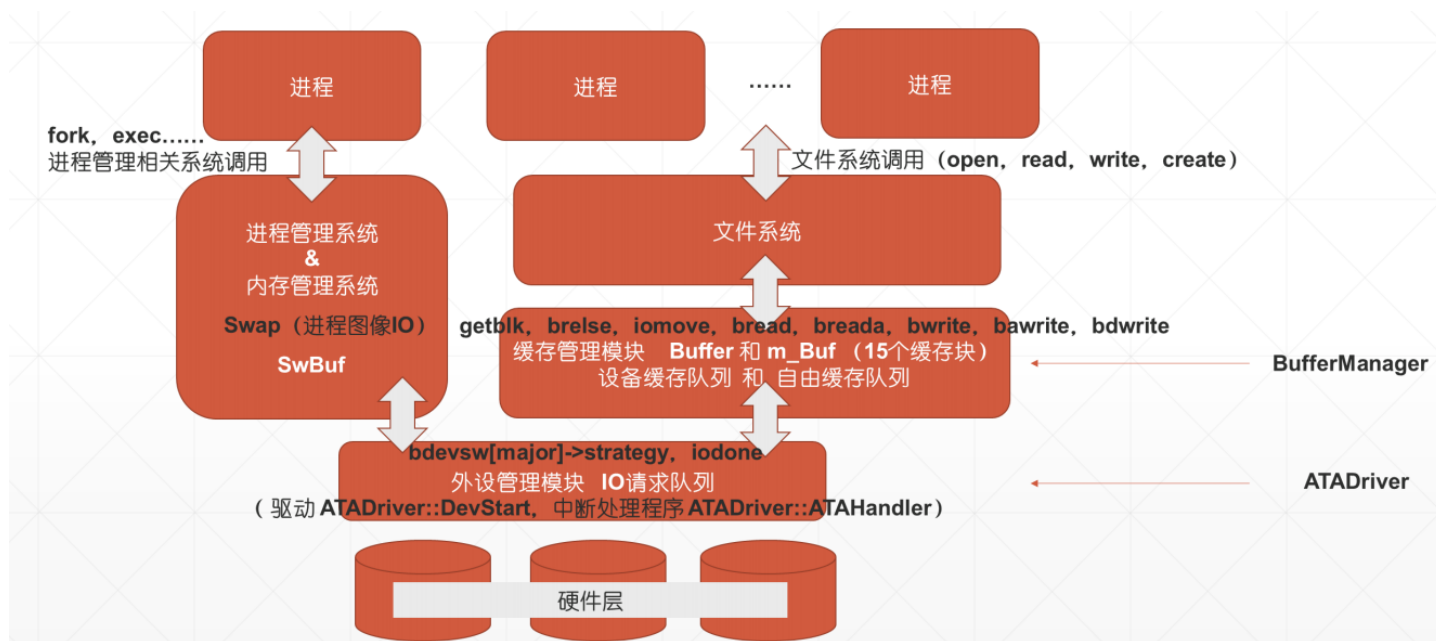
PRD表（Program Request Descriptor Table）通常用于描述和管理与I/O操作相关的请求和设备状态。

PRD表通常会包含以下几个字段：

缓冲区地址、缓冲区大小、I/O请求类型、设备ID、状态字段、优先级。

PRD表的一个重要应用是在DMA（直接内存访问）操作中。在使用DMA时，PRD表记录了DMA的操作参数，包括源地址、目标地址、传输的字节数等信息。DMA控制器根据PRD表中的信息执行数据传输操作，而不需要CPU直接干预。

在这种情况下，PRD表起到了**将内存空间和外部设备的I/O请求进行映射**的作用，确保DMA能够按要求完成数据传输。



外设管理例题1

例题 1、

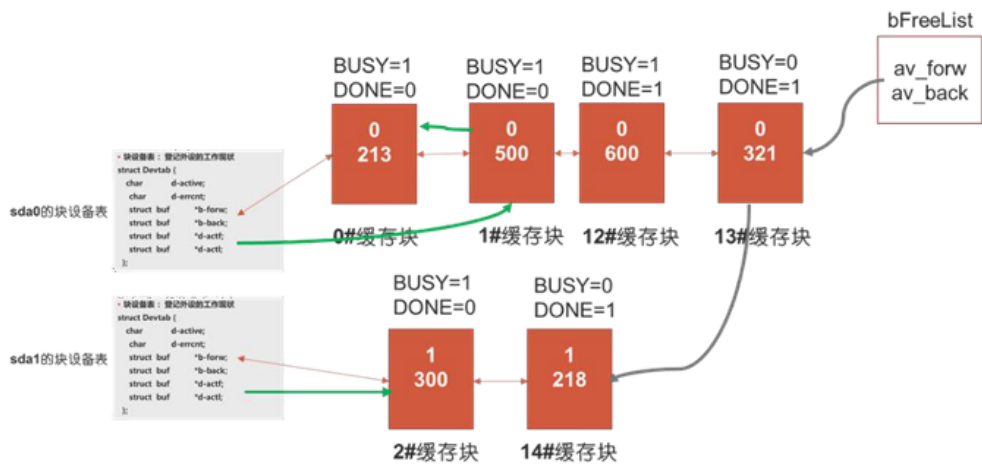


图 3、缓存队列。灰色，自由缓存队列；红色，设备缓存队列；绿色，IO 请求队列

【参考答案】

1、写出系统的自由缓存队列，磁盘sda0， sda1 的设备缓存队列和 IO 请求队列

- 自由缓存队列: $\langle 0, 321 \rangle \rightarrow \langle 1, 218 \rangle$
- sda0 的设备缓存队列: $\langle 0, 213 \rangle \rightarrow \langle 0, 500 \rangle \rightarrow \langle 0, 600 \rangle \rightarrow \langle 0, 321 \rangle$
- sda0 的 IO 请求队列: $\langle 0, 500 \rangle \rightarrow \langle 0, 213 \rangle$
- sda1 的设备缓存队列: $\langle 1, 300 \rangle \rightarrow \langle 1, 218 \rangle$
- sda1 的 IO 请求队列: $\langle 1, 300 \rangle$

2、sda0 正在进行的读操作:

- 内存地址: Buffer[1]
- 扇区: 500
- IO 完成时，磁盘中断处理程序
 - 会启动新的 IO 请求吗? 会，IO 213#扇区
 - 会唤醒进程吗? 会，因为读操作是同步的。

3、sda1 正在进行的写操作:

- 内存地址: Buffer[2]
- 扇区: 300
- IO 完成时，磁盘中断处理程序
 - 会启动新的 IO 请求吗? 一定不会，执行完此 IO 后 IO 请求队列为空
 - 会唤醒进程吗? 一般不会，因为通常基于缓存的写操作是异步的。但如果有进程睡眠等待自由缓存，唤醒它。牢记第一种，通常的情况。

0: b_dev 设备号 500: blkno: 扇区号

缓存控制块:

例题 2：在 UNIX V6++ 中，试说明缓存控制块 Buf 有无可能，在什么样的条件下出现下列情况：

- (1) 同时处在自由 Buf 和一个设备 Buf 队列中；
可能, Brelse 释放的自由缓存块
- (2) 同时处在某一设备 Buf 队列和 I/O 请求队列中；
可能, IO 操作尚未完成的所有缓存块都是这样的
- (3) 只处在某一设备Buf 队列中；
可能。IO 完成 (B_DONE 是 1) 但 IOMove 还没有完成的缓存块。
- (4) 只处在 I/O 请求队列中，不在设备缓存队列里。
可能。负责传送进程图像的 SwBuf。
- (5) 同时出现在自由 Buf 队列和某个设备的 I/O 请求队列中
不可能, 自由缓存不可能在 IO 请求队列，反之亦然。
- (6) 同时出现在一类设备的 Buf 队列、另一类设备的 I/O 请求队列中；
不可能, 一个缓存块不可能分配给 2 个设备。

(1)

释放数据块的使用权。Brelse(Buf* bp)



```
void BufferManager::Brelse(Buf* bp)
{
    .....
    if(bp->b_flags & Buf::B_WANTED)
    {
        procMgr.WakeupAll((unsigned long)bp); // 唤醒等待使用数据块的进程
    }
    if(this->bFreeList.b_flags & Buf::B_WANTED)
    {
        this->bFreeList.b_flags &= (~Buf::B_WANTED);
        procMgr.WakeupAll((unsigned long)&this->bFreeList); // 唤醒等待自由缓存的进程
    }
    .....
    X86Assembly::CLI(); // 缓存块放队尾
    bp->b_flags &= ~(Buf::B_WANTED | Buf::B_BUSY | Buf::B_ASYNC);
    (this->bFreeList.av_back)->av_forw = bp;
    bp->av_back = this->bFreeList.av_back;
    bp->av_forw = &(this->bFreeList);
    this->bFreeList.av_back = bp;
    X86Assembly::STI();
    return;
}
```

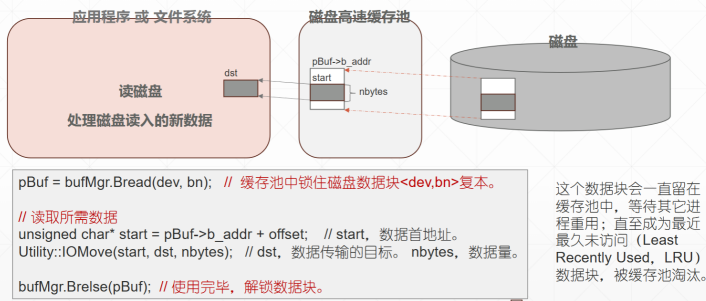
(3) 先字符块同步读入缓存Bread，这一步算IO完成

Bread里有Getblk，在这里插入设备队列

然后还要传给应用程序，这一步算IOMove

结束了才能从设备释放缓存块

7.2 同步读 (Unix 基本读操作)



bp = Getblk(dev, blkno)

遍历dev的设备缓存队列

- 搜索缓存池, 寻找分配给 < dev, blkno > 的缓存块 (缓存控制块 b_dev==dev, b_blkno==blkno)
- 找到 (缓存命中, Buffer[i])
- 没找到 (缓存不命中), 为 < dev, blkno > 分配缓存块 Buffer[j].

```
loop:
.....
X86Assembly::CLI();
if(bp->b_flags & Buf::B_BUSY)
{
    bp->b_flags |= Buf::B_WANTED;
    .....Sleep(unsigned long)bp, PRIBIO);
    X86Assembly::STI();
    goto loop;
}
this->NotAvail(bp);
return bp;
```

```
loop:
.....
if(this->bFreeList.av_forw == &this->bFreeList)
{
    this->bFreeList.b_flags |= Buf::B_WANTED;
    .....Sleep(&this->bFreeList, PRIBIO);
    goto loop;
}
bp = this->bFreeList.av_forw;
this->NotAvail(bp);
{
    bp->b_flags |= Buf::B_ASYNC;
    this->Bwrite(bp); // 异步写回磁盘
    goto loop;
}
```

4) 进程图像传送请求块SwBuf, 在这里插入了IO请求队列

7.1.1 ATA硬盘驱动程序

缓存层或进程管理模块发出IO请求。bp是一个 m_buf[i] 或 Swbuf。

```
int ATABlockDevice::Strategy(Buf* bp)
{
    .....
    bp->av_forw = NULL;

    X86Assembly::CLI();

    IO控制块插入IO请求队列尾部;
    if(this->d_tab->d_active == 0)
    { /* 原先磁盘空闲, 立即发IO命令 */
        this->Start();
    }

    X86Assembly::STI();

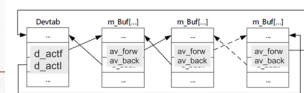
    return 0; /* GCC likes it ! */
}

void ATABlockDevice::Start() // 启动IO请求队列队首IO操作
{
    Buf* bp;

    if( (bp = this->d_tab->d_actf) == 0 )
        return; /* IO请求队列空, 返回 */

    /* IO请求队列不空, 设置控制器忙标志 */
    this->d_tab->d_active++;
    /* 编程硬件, 启动IO操作 */
    ATADriver::DevStart(bp);
}
```

IO请求队列



例题 3、T0 时刻，PA、PB 进程先后访问文件A， PA read 4#字节， PB write 4#字节。已知文件 A 的 0#逻辑块（文件的 0#~511#字节） 存放在 666#扇区。T0 时刻缓存不命中。自由缓存队列不空，所有自由缓存不脏（不带延迟写标识），队首缓存块 Buffer[7]。

1、请分析如下时刻进程 PA，PB 的调度状态 和 Buffer[i]的使用状态。

- PA 执行read 系统调用。

缓存不命中。Buffer[7]是 LRU 自由缓存，且不脏，分配用来装新的数据块（666#扇区）。

刷新 m_buf[7]: dev=0, blkno=666, B_READ=1, B_DONE=0, 送 IO 请求队列之后，PA 执行 IOWait 函数入睡：

- sleep(& m_buf[7], -50), 高优先权睡眠。等待 IO 完成，也就是 m_buf[7]的 B_DONE变 1。

```
while( (bp->b_flags & Buf::B_DONE) == 0 )  
    u.u_procp->Sleep((unsigned long)bp, -50);
```

Buffer[7]状态：上锁(B_BUSY==1)，在 IO 请求队列；数据不可用 (B_DONE==0)。进程 PA 持锁。

- PB 执行 write 系统调用

缓存命中，PB 执行 GetBlk 函数时入睡：

- sleep(& m_buf[7], -50), 高优先权睡眠。等待持锁进程PA 执行 Brelse()解锁，也就是 Buf[7] 的 B_BUSY 变 1。B_WANTED 置 1。

```
while( (bp->b_flags & Buf::B_BUSY) == 0 ) {  
    bp->b_flags |= Buf::B_WANTED  
    u.u_procp->Sleep((unsigned long)bp, -50); }
```

Buffer[7] 状态：上锁(B_BUSY==1)，在 IO 请求队列；数据不可用 (B_DONE==0)。进程 PA 持锁。有进程等待使用其中的数据 (B_WANTED==1)。

- 666#扇区 IO 完成

中断处理程序执行 IODone()函数，m_buf[7] B_DONE=1 置 1，唤醒PA 和PB。PA、PB 就绪，之后依次上台运行。

Buffer[7] 状态：上锁(B_BUSY==1)，不在 IO 请求队列，在设备缓存队列，不在自由缓存队列；数据可用 (B_DONE==1)。进程 PA 持锁。有进程等待使用其中的数据 (B_WANTED==1)。

- 若 PA 先上台运行。系统先后发生以下行为。

- ◆ B_DONE==1，PA 执行 IOmove 将 4# 字节复制进用户空间。解锁缓存 (B_BUSY=0, B_WANTED=0)，送自由缓存队列队尾，唤醒PB；
- ◆ PB 上台运行。B_BUSY==0，PB 锁住缓存，先读后写，将新数据写入 Buffer[7] 4#字节后，置延迟写标记（脏标记）B_DELWR。解锁缓存，标记缓存块中数据可用（是最新版本）B_DONE=1，送自由缓存队列队尾。（没 IO 的）

- 若 PB 先上台运行。系统先后发生以下行为。

- ◆ B_BUSY==1。PB，sleep(& m_buf[7], -50)再次入睡。置 1 Buf[7]的 B_WANTED 标识。
 - Buffer[7] 状态：上面绿的。
- ◆ PA 上台运行。B_DONE==1，PA 执行 IOmove 将 4#字节复制进用户空间。解锁缓存 (B_BUSY=0, B_WANTED=0)，送自由缓存队列队尾，唤醒PB。
 - ◆ 上面紫的。

- PA 进程能够读到PB 进程写入的新数据吗？

不能。因为，IO 完成时，一定是 PA 先使用缓存块中的数据。

B_WANTED：有进程正在等待使用该buf管理的资源

PA持锁，只有PA能解锁，所以PB会入睡。

例题 4：进程读磁盘文件，存在不入睡的可能吗？何时？ 存在。需要读入的数据命中自由缓存。

例题 5：时刻 T1，数据块 556、600、782、891、900 缓存不命中。已知：将IO请求放入IO请求队列后至IO操作完成，进程平均需要等待 T。从缓存复制数据到用户空间，平均耗时 t。T>>t。请比较使用缓存池的IO 和 未使用缓存池的IO，系统的性能。指标1，IO请求的平均耗时；指标2，完成整个IO请求序列系统的总耗时。

1、读操作序列：556, 556, 556, 600, 782, 891, 900, 556, 556, 556

不使用缓存池 数据从磁盘直接复制到用户空间，IO请求平均耗时 T。完成所有读操作，IO 10次，系统的总耗时 10T。

使用缓存池 数据从磁盘复制到缓存。之后进程从缓存中取用数据，复制到自己的用户空间。完成整个序列，IO 5次。总耗时，5T+10t。IO请求平均耗时 T/2+t。

磁盘高速缓存的作用 重用IO数据。缩短读操作平均耗时。减少 IO 请求次数，有利于缩短 IO 操作的平均耗时，提高IO子系统的吞吐率。

- 对于数据块 556，第一次读取时缓存不命中，会产生一个 IO 请求将其从磁盘复制到缓存。之后的 3 次读取 556 都命中缓存，不需要再次从磁盘读取，所以 556 只产生了 1 次 IO 请求。
- 同理，600、782、891、900 第一次读取时缓存不命中，各产生 1 次 IO 请求，总共 4 次。所以完成整个读操作序列，IO 请求次数为 5 次。
- 每次从缓存复制数据到用户空间平均耗时 t，读操作序列有 10 次从缓存复制数据到用户空间的操作（每个数据块读取都需要复制到用户空间），所以复制数据到用户空间的总耗时为 10t。

外设管理例题2

记住写到最后一个字节，缓存块要异步写回磁盘。

类似刚刚的分析：

二、T1 时刻缓存不命中。自由缓存队列不空，所有自由缓存不脏（不带延迟写标识），队首缓存块 Buffer[7]。PA、PB、PC 进程先后访问文件A，已知该文件0#逻辑块（0~511字节）存放在55#扇区。请分析如下时刻进程调度状态 和 Buffer[i]的使用状态。

（1）PA、PB、PC 进程先后访问文件A，PA read 4#字节，PB write 200#字节，PC read 500#字节。

（2）PA、PB、PC 进程先后访问文件A，PA read 4#字节，PB write 511#字节，PC read 500#字节。

答：

（1）

- PA 执行read 系统调用

- 调度状态。PA 为 55# 磁盘数据块申请得到一块缓存，发起读 IO 请求后，sleep(&m_buf[7], -50)，入睡等待 IO 完成、B_DONE 变为 1。
- Buffer[7]的使用状态：分配给 55#磁盘数据块，在 IO 请求队列。B_BUSY 为 1，B_DONE 为 0。

- PB 执行 write 系统调用

- 调度状态。PB 等待复用 55#数据块，sleep(&m_buf[7],-50)，入睡等待缓存块解锁、B_BUSY 变为 0。
- Buffer[7]的使用状态：分配给 55#磁盘数据块，在 IO 请求队列。B_BUSY 为 1，B_DONE 为 0，B_WANTED 为 1。

- PC 执行 read 系统调用

- 同 PB。

- 55#扇区 IO 完成

调度：中断处理程序执行 IODone()函数，m_buf[7] B_DONE=1 置 1，唤醒 PA，PB 和 PC。

Buffer[7] 的使用状态：上锁(B_BUSY==1)，不在 IO 请求队列，在设备缓存队列，不在自由缓存队列；数据可用 (B_DONE==1)。进程 PA 持锁。有进程等待使用其中的数据 (B_WANTED==1)。

调度：PA 先用缓存块中的数据，执行 IOmove 将 4#字节复制进用户空间。解锁缓存 (B_BUSY=0, B_WANTED=0)，送自由缓存队列队尾，唤醒 PB 和 PC。PB 和 PC 依次上台，分别执行 write 系统调用和 read 系统调用的下半段，写、读缓存块中偏移量依次为 200 和 500 的字节。谁先执行，不得而知，对最终结果没有影响。

Buffer[7] 的最终状态：B_DELWR==1, B_DONE==1, B_BUSY==0，是一块自由的脏缓存。

（2）和第一小题一致。唯一的不同在于，本小题 PB 写至数据块底部，55#数据块需要异步写回磁盘。若其先于 PC 执行，PC 会再次入睡，等待中断处理程序解锁 55#数据块。

Buffer[7] 的最终状态：B_DELWR==0, B_DONE==1, B_BUSY==0，是一块自由的干净的缓存。

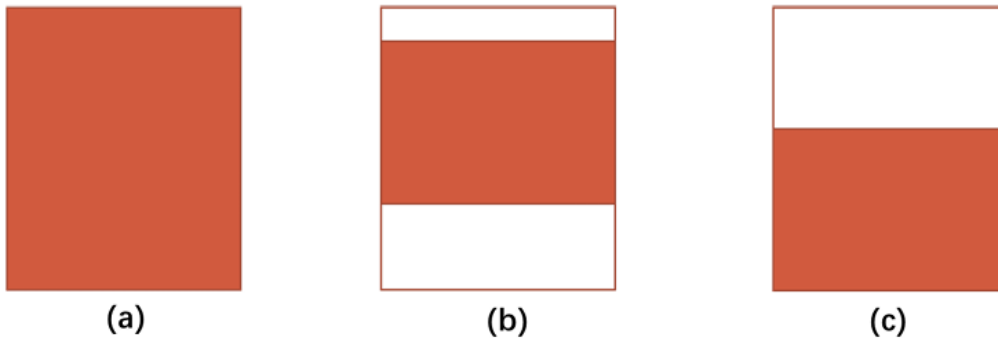
一、不借助缓存写磁盘，需要先读后写吗？

答：需要。这是因为扇区是磁盘读写的原子单位。写命令会更新整个扇区（512 字节）。所以，当写入的字符量不足 512 字节时，必须先读以保护不被写操作改写的区域。

二、针对下图中的所有情况，写数据块进程是否会入睡，需要执行几次 IO 操作？假设系统自由缓存队列中存在不脏的自由缓存。

1、目标磁盘数据块缓存不命中

2、目标磁盘数据块缓存命中



答：

- (1) 缓存块不命中。需要为数据块分配缓存块，自由缓存队列非空，进程不会睡。
 - 子图 (a)，无需先读，不延迟写。一次异步写IO，进程不睡。
 - 子图 (b)，需要先读，延迟写。一次同步读 IO，没有写IO。进程会睡，等待读操作结束。
 - 子图 (c)，需要先读，不延迟写。两次 IO 操作，一次同步读，一次异步写。进程等待读操作结束时会睡。
- (2) 缓存块命中。不需要为数据块分配缓存块，但是需要与其它进程竞争缓存块的使用权，其间进程可能会入睡等待缓存块解锁，排除这种情况。
 - 子图 (a)，无需先读，不延迟写。一次异步写IO。进程不睡。
 - 子图 (b)，需要先读，延迟写。没有 IO 操作。进程不睡。
 - 子图 (c)，需要先读，不延迟写。一次异步写 IO。进程不睡。

注：这里的先读指的是执行 `Bread()` 函数。

红色块：写入的新数据

三、读写序列：556（写 40#字节），556（写 1#字节），556（写 0~127#字节），600，782，891，900，556（写 128~255#字节），556（写 256~383#字节），556（写 384~511#字节）。
请问，完成序列中556#数据块全部写操作，需要执行几次磁盘 IO ？总耗时多少？平均耗时？

(1) 不使用磁盘高速缓存。已知发出 IO 请求至 IO 操作完成，进程平均需要等待 T。

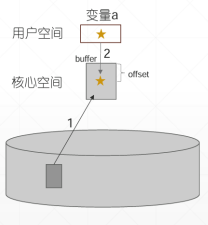
【参考答案】每次写556#扇区均先读后写。6*2=12次IO。总耗时12*T+6t；平均耗时2*T+t。。

(2) 使用高速缓存。已知自由缓存队列中有足够多的干净缓存。将IO 请求放入 IO 请求队列后至 IO 操作完成，进程平均需要等待 T。从缓存复制数据到用户空间（含上锁、解锁操作），平均耗时 t。

【参考答案】2 次 IO。第一次写 556#块的时候，先读会 IO，最后一次写 556#块的时候，写满了，一次异步写 IO。总耗时 2*T+6t；平均耗时 T/3+t，远远小于不使用磁盘高速缓存的情形。

每次写都要先读后写原因：

借助缓存的写操作（不考虑并发）

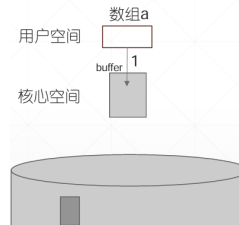


内核，应用程序变量a写盘（写入数据量不足一个数据块，先读后写）

- 计算数据data所在的磁盘数据块号blkno，找到分配给blkno的缓存块buffer
- (1) if(buffer尚未与blkno同步，数据不可用)
启动IO操作读入磁盘数据块，入睡等待IO完成
- 计算data在缓存块中的偏移量offset
- (2) 将变量a复制到缓存块中（送数据至缓存，数据传输）
buffer[offset] = a;
- 结束

操作系统(侯宏斌 / 顾毅 2024) 同济大学计算机系 邓睿 9

借助缓存的写操作（不考虑并发）



内核，应用程序变量a写盘（写入一整块，无需先读）

- 计算数据data所在的磁盘数据块号blkno，找到分配给blkno的缓存块buffer
- 将用户空间数据块写进缓存块
buffer = 数组a;
- 结束

6是复制到用户空间，也就是读的次数

写满的原因：

```
class Buf {
// 管理缓存块：使用状态和 IO 参数。
public:
    unsigned int b_flags; // 位图，登记缓存使用状态
    int padding; // 技术细节，看代码

    Buf* b_forw; // 队列指针，用来管理缓存
    Buf* b_back;
    Buf* av_forw;
    Buf* av_back;

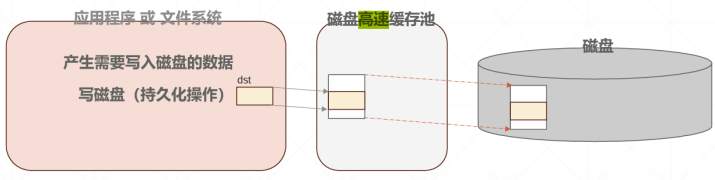
    unsigned char* b_addr; // Buffer[] 首地址
    short b_dev; // 磁盘号
    int b_blkno; // 扇区号
    int b_wcount; // IO 参数，传送字节数
    int b_error; // IO 错误信息
    int b_resid; // IO 出错时还未传送的剩余字节数
};

// 设置一个 PRD
prd.SetBaseAddress((unsigned long)bp->b_addr & ~0xC0000000); // 首地址（物理地址）
prd.SetByteCount(bp->b_wcount); // 文件系统数据块 512 字节；进程图像，任意长度
table.SetPhysicalRegionDescriptor(0, prd, true); // 写进 PRD 表，第 0 项

DMA::Reset(); // 复位 DMA 控制器
```

操作系统(侯宏斌 / 顾毅 2024) 同济大学计算机系 邓睿 31

7.3 异步写（Unix 基本写操作）



应用程序或文件系统 产生需要写入磁盘的数据 写磁盘（持久化操作）

磁盘高速缓存池 磁盘

数据写入缓存池后，解锁缓存块，写操作完毕。数据块可供其它进程使用。需要注意的技术细节：

- 如果写入磁盘数据块的数据量不足512字节。先读后写，保护磁盘数据块中新数据覆盖不到的旧数据。
- 数据块写满时刷回磁盘。这是假定进程会顺序写磁盘数据，数据块写至底部不再有重用价值，异步写回磁盘。
 - 若写不满，打脏标识、解锁缓存块，写操作完毕。

一、看图，回答问题



(1) 图中为什么没有 IO 请求队列？

答：所有缓存块 `B_DONE` 是 1，IO 已完成。IO 请求队列是空的。

(2) 读 213, 500, 600, 321 块。不管是否脏缓存（是否有延迟写标记），进程不会睡，没 IO。

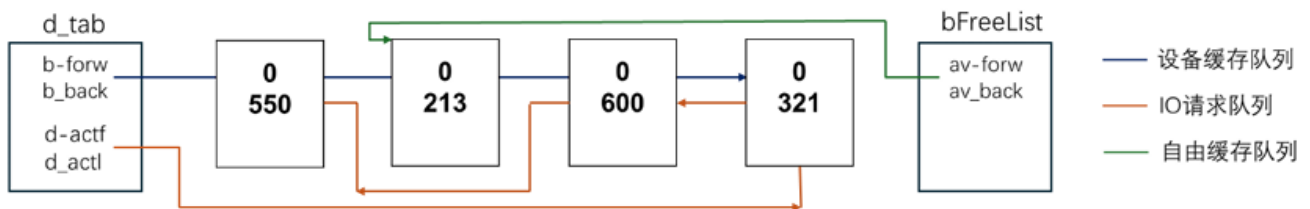
(3) 写 213, 500, 600, 321 块。不管是否脏缓存（是否有延迟写标记），进程不会睡。若缓存写满，异步写回磁盘，一次 IO。没写满，没有 IO。

2、3，注意自由缓存队列的变化。GetBlk，锁住某个缓存块之后，会把它从自由缓存队列里抽出来。Brelse 释放缓存块的时候，送自由缓存队列末尾。

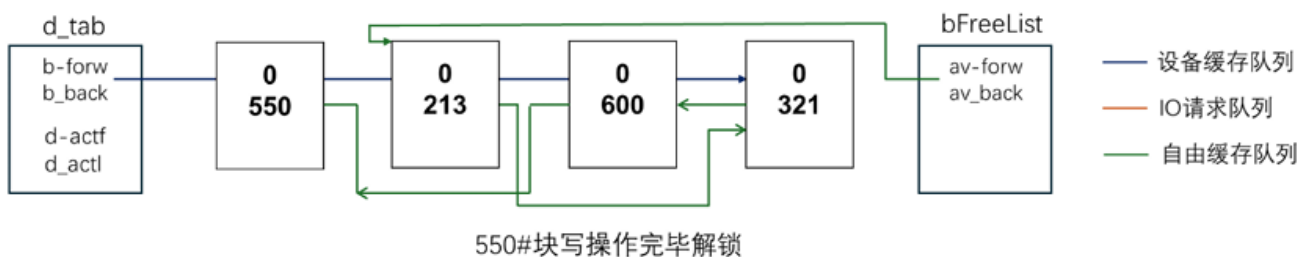
(4) 写 550#块，偏移量为 8 的字节。

答：缓存不命中。分配自由缓存队列第一个不脏的 Buffer（不带延迟写标记 `B_DELWR`）：将沿途所有脏缓存（321, 600 号数据块）异步写回磁盘，为 550#数据块分配 500#数据块原先占据的缓存块。先读后写，一次同步读 IO。如图：

- 由于 I/O 操作已经完成，不需要等待读操作的执行，因此进程不会进入等待状态（不会“睡”）。



3个IO请求块送入IO队列。设备缓存队列队首是最后发出的IO请求。脏缓存块在队列中位置不变（考试不要求）。



外设管理习题

期末考字符设备的缓存不要深入探究，看懂那两页PPT（哪两页你这煞笔

文件管理

Part4 外设和文件3

- 文件按磁盘数据块大小切块，离散存放在磁盘上。每一块是一个逻辑块。
- 为文件分配的磁盘存储空间，是整数个磁盘数据块。
- 文件系统使用离散存储管理方式。
 - 相邻的文件逻辑块在磁盘上不必连续存放。但尽量连续存放一定能提升文件顺序读写效率。

文件控制块（FCB）

每个文件有一个文件控制块。存有：

1. 文件名，文件尺寸。
2. 地址映射表，用来计算分配给逻辑块bn的物理块号dn。使用方法：

$dn = f(bn); /* bn = offset/512; offset是数据在文件中的偏移量 */$

每个块512字节。

3. 文件主 uid，创建该文件的进程的uid。

三、文件控制块（File Control Block，简称FCB）

每个文件有一个文件控制块

- 文件名，文件尺寸。
- 地址映射表，用来计算分配给逻辑块bn的物理块号dn。使用方法：
 - $dn = f(bn); /* bn = offset/512; offset是数据在文件中的偏移量 */$
 - 数据存放在物理块dn，块内偏移量是 $offset \% 512$
- 文件主 uid，创建该文件的进程的uid。
- 文件访问权限，登记各类用户对该文件的读、写、执行权限。
 - Unix 是 `RWXRXRXRX`，标识文件主，文件所在的用户组和nobody对文件的读、写、执行 权限。
 - Windows 是 ACL（Access Control List）。
- 进程 uid 和 gid 标识使用文件的用户 和 用户组
- 创建时刻，最后一次访问时刻，最后一次修改时刻。

Unix 系统，文件控制块存放在 2 处：

文件名存放在目录项（DirectoryEntry）

其余属性存放在索引节点（inode，i 是index 索引的缩写）。



文件树

Unix的目录文件



DiskNode 索引节点区

文件索引节点以静态方式存储在磁盘的一个专门区域，称为**外存索引节点 (DiskNode) 区**，保存所有文件索引节点按顺序从0开始编号。

```
class DiskNode {
    unsigned int d_mode;
    int d_nlink; /* 硬链接数 */
    short d_uid; /* 文件主的uid */
    short d_gid; /* 文件所在的组 */
    int d_size; /* 文件尺寸，字节数 */
    int d_addr[10]; /* 混合索引表根节点 */
    int d_atime; /* 最后访问时刻 */
    int d_mtime; /* 最后修改时刻 */
};
```

DiskNode是Unix的索引节点

文件卷格式

Unix V6文件卷分 3 个区域：超级块、inode区和 数据区



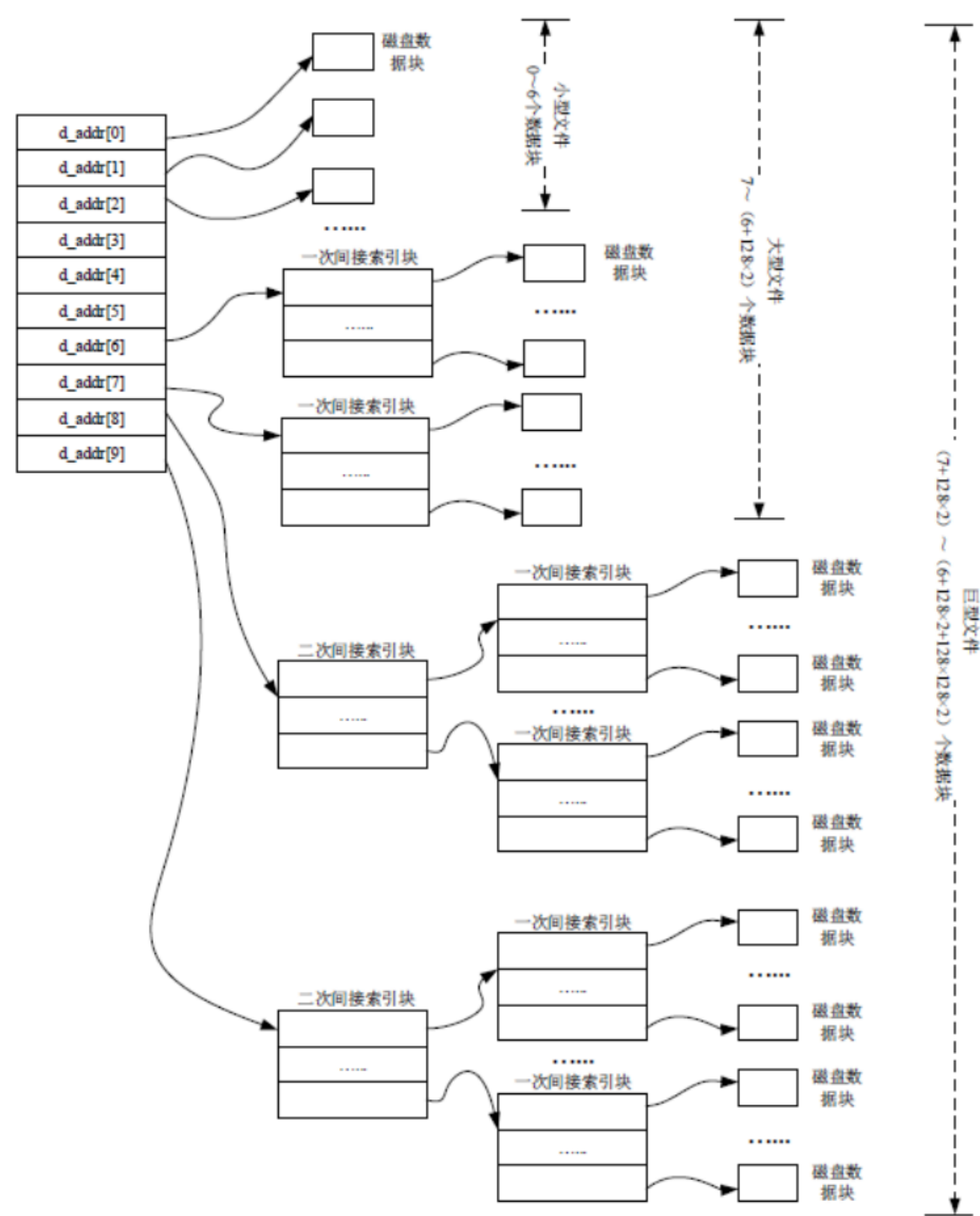
- 超级块，存放文件卷 info 和文件系统使用状态
- inode区存放DiskNode
- 数据区存放 普通数据文件 和 目录文件（数据块、索引块）

文件索引结构

由DiskNode中的d_addr[10]决定数据索引位置。具体来说：

1. **小型文件索引**：d_addr[0] ~ d_addr[5]，可以登记6个物理块号，存放文件的0#~5#逻辑块。
2. **大型文件索引**：d_addr[6] ~ d_addr[7]，指向**索引盘块**，每个索引盘块可以登记 **128** 个物理块号。

3. 巨型文件索引：d_addr[8] ~ d_addr[9]，指向2次索引块。每个2次索引盘块可以指向128个一次索引盘块。



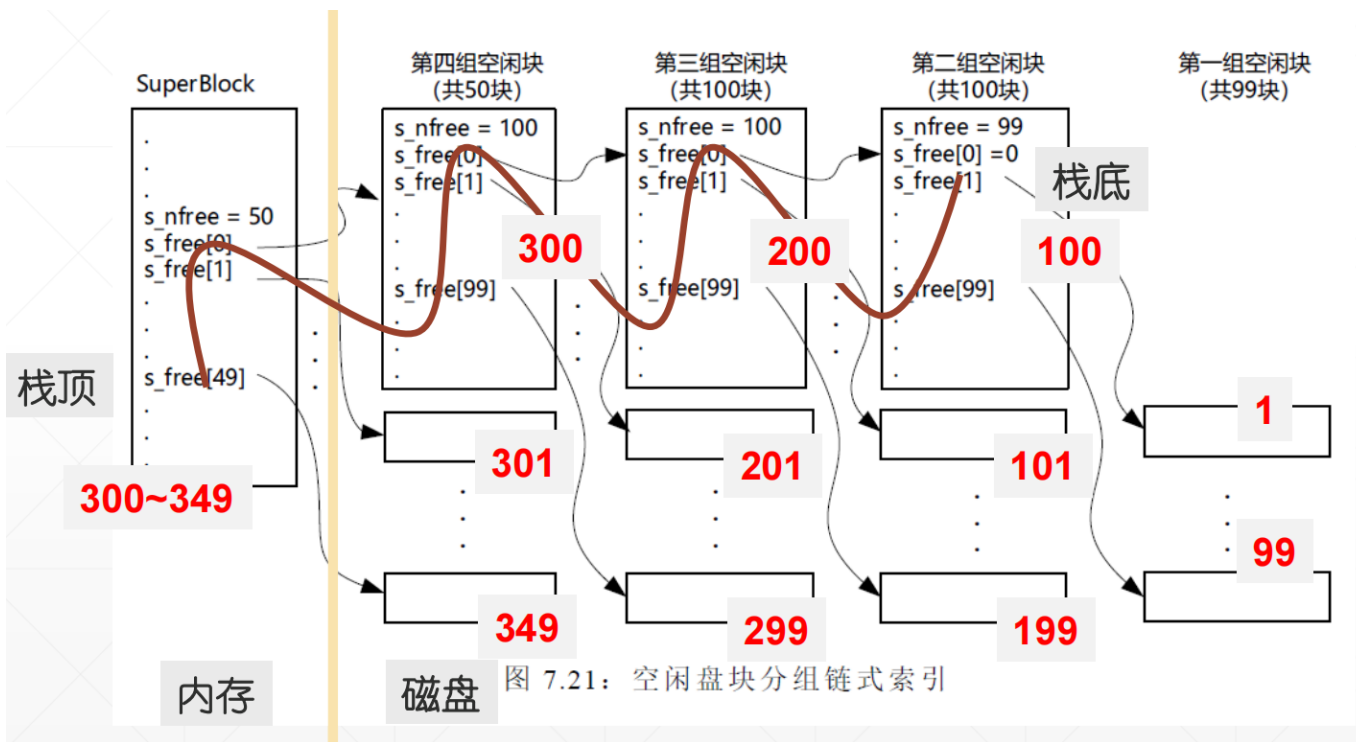
空闲盘块管理

用空闲扇区登记其余空闲扇区号。

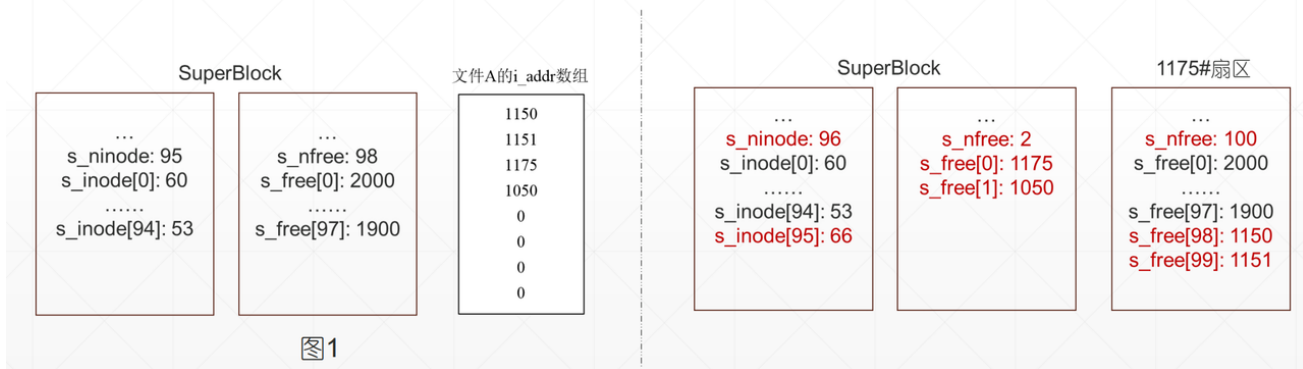
每100个空闲扇区编成一组。最先入栈的空闲扇区是队长，用于登记前一组所有空闲扇区号。

第一组，组长盘块空缺，99个盘块。

最后一组，SuperBlock中的s_free[0]~s_free[n]指向，s_nfree是指向的空闲组的空闲盘块数量。



- 某Unix V6++文件卷，SuperBlock状态如下图1。66#文件A，混合索引表状态如下图2。请图删除文件A后，SuperBlock的状态变化。



Part4 外设和文件4

内存索引节点表 InodeTable

▪ 内存Inode表

g_InodeTable

每个非空元素是一个内存Inode，管理一个使用中的文件，装有这个文件的DiskInode。

```
class InodeTable
{
public:
    static const int NINODE = 100; // 100个内存Inode
public:
    Inode m_Inode[NINODE]; // 内存Inode表
}
```

▪ 系统打开文件表

g_OpenFileTable

每个非空元素是一个File结构，管理一次文件系统会话。

```
class OpenFileTable
{
public:
    static const int NFILE = 100; // 100个File结构
public:
    File m_File[NFILE]; // 系统打开文件表
};
```

File类

```
class File
{
public:
    enum FileFlags
    {
        FREAD = 0x1, /* 读操作 */
        FWRITE = 0x2, /* 写操作 */
        FPIPE = 0x4, /* 管道文件读写 */
    };
public:
    unsigned int f_flag; // 文件的操作方式。FREAD、FWRITE 或 FREAD | FWRITE
    int f_count; // 使用该File结构的进程数量
    Inode* f_inode; // 指向目标文件的内存Inode
    int f_offset; // 读写指针
};
```

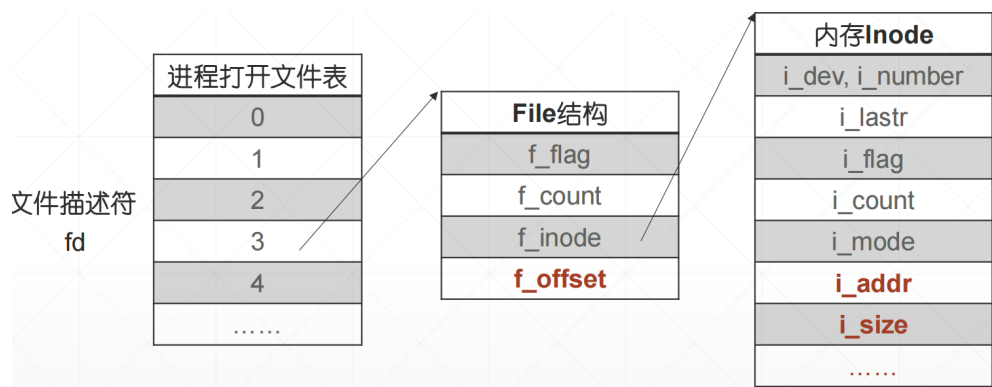
1、T0 时刻，系统中有两个进程 P1 和 P2，分别独立打开并同时访问小文件 example。则在内存打开文件结构中有（ A ）个内存 i 节点指向该文件？（ B ）个 file 结构记录着进程对文件的访问情况？

A. 1 B. 2

OpenFile

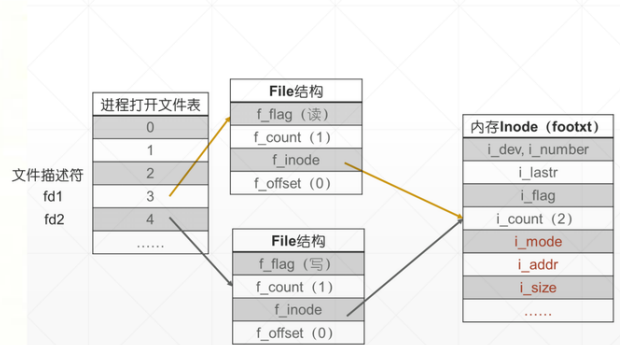
ProcessOpenFileTable进程打开的文件表

```
class OpenFiles
{
public:
    static const int NOFILES = 15;
private:
    File *ProcessOpenFileTable[NOFILES]; // 15个元素
};
```



```
int main()
{
    int fd1, fd2;
    char c;

    fd1 = Open("foobar.txt", O_RDONLY, 0);
    fd2 = Open("foobar.txt", O_RDONLY, 0);
    Read(fd1, &c, 1);
    Read(fd2, &c, 1);
    printf("c = %c\n", c);
    exit(0);
}
```



在哪个数据结构中登记有进程对文件的访问方式（读或读写）？ (B)

文件的读写指针保存在 (B) ？

组成文件的每个逻辑盘块（信息块）在磁盘上的地址保存在 (A C) ？

A. 内存 i 节点 B. file 结构 C. i_addr 数组

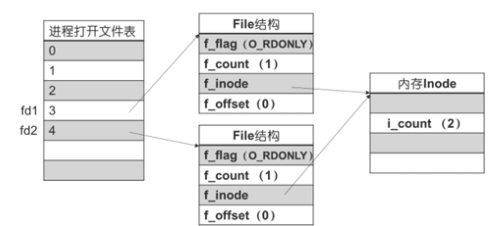
2、假设 foobar.txt 文件的内容是字符串“1234567890”。请问（1）这个程序的输出是什么？（2）画进程的打开文件结构

```
int main()
{
    int fd1, fd2;
    char c;

    fd1 = Open("foobar.txt", O_RDONLY, 0);
    fd2 = Open("foobar.txt", O_RDONLY, 0);
    Read(fd1, &c, 1);
    Read(fd2, &c, 1);
    printf("c = %c\n", c);
    exit(0);
}
```

答：输出 1。

打开文件结构：



3、fd1, fd2 的值是几？ fd1==fd2==3

```
int main()
{
    int fd1, fd2;

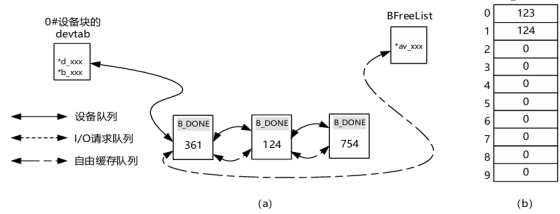
    fd1 = Open("foo.txt", O_RDONLY, 0);
    Close(fd1);
    fd2 = Open("baz.txt", O_RDONLY, 0);
    printf("fd2 = %d\n", fd2);
    exit(0);
}
```

文件系统例题和习题

文件系统情景分析题 1

完整的文件读写过程

```
int fd = open("/usr/ast/Jerry", 3); //以可读可写方式打开文件
char data[300];
seek(fd, 500, 0); //将文件读写指针定位到第500字节
int count = read (fd, data, 300); //从文件读300字节到data
count = write(fd, data, 300); //从data 写 300 字节到文件
```



缓存队列 和 Jerry 文件的索引表

一、open 完成后 i_lastr == -1； seek 将 f_offset 调成 500

二、read 系统调用的执行过程：

```
count = read (fd, data, 300)
```

1、f_offset==500，文件长度 750。read 系统调用从文件中读 250 字节至 data 数组：data[0]~data[249]。data 数组是局部变量，故后 50 个字符是乱码。

2、细节：要读 0#、1#逻辑块，分 2 次完成。

2.1 初始化 IO 参数：m_offset = f_offset = 500，m_base = data，m_count = 300

2.2 读 0#逻辑块：

当前逻辑块 bn = m_offset/512 = 0，块内偏移量 500，读 12 字节至块结束。

查混合索引表，得当前块 bn 的物理块号 123，下一块的物理块号 124。

$bn = i_lastr + 1$ ，顺序读，执行预读操作 `breada (123, 124)`，当前块 123 缓存不命中，预读块 124 缓存命中，PA 为当前块构造 IO 请求块，睡眠等待 IO 完成。

123#块 IO 完成后，中断处理程序唤醒 PA 进程。PA 进程执行 `IOmove` 函数将缓存块中 500# ~ 511#字节复制到用户空间 `data[0]~data[11]`。缓存使用完毕，解锁。

修改 IO 参数， $m_offset = 512$ ， $m_base = data + 12$ ， $m_count = 300 - 12$ 非 0 并且文件没结束，继续读下一块。

2.3 读 1#逻辑块：

当前逻辑块 $bn = m_offset / 512 = 1$ ，块内偏移量 0，读 238 个字节。

查混合索引表，得当前块 bn 的物理块号 124。文件结束，没有下一块。

$bn = i_lastr + 1$ ，顺序读，`breada (124, 0)`。124#物理块缓存命中，锁住，复用这块缓存中的数据。`iomove` 将该缓存中 0#~237#字节复制到用户空间 `data[12] ~ data[249]`，解锁缓存。预读块号为 0，表示无需执行预读操作。

修改 IO 参数， $m_offset = 512 + 238 ==$ 文件长度 i_size 。读操作结束，修改读写指针 $f_offset = 750$ 。返回实际读入的字节数 250。

三、write 系统调用的执行过程：

```
count = write(fd, data, 300)
```

1、 $f_offset == 750$ ，文件长度 750。write 系统调用将 `data` 数组中的 300 个字节追加写在文件尾部。写操作完成后，文件长度增加至 1050。

2、细节：分 2 次写入，要为 2#逻辑块分配新数据块。

2.1 初始化 IO 参数： $m_offset = f_offset = 750$ ， $m_base = data$ ， $m_count = 300$

2.2 写 1#逻辑块：

当前逻辑块 $bn = m_offset / 512 = 1$ ，块内偏移量 238，写 274 字节至块结束。

查混合索引表，得当前块 bn 的物理块号 124。

先读后写，这是因为写入的字节数 274 不足 512 字节。124#物理块缓存命中，先读操作只需锁住该缓存块。

`iomove` 将用户空间中的数据 `data[0] ~ data[273]` 写入这块缓存。

修改 IO 参数， $m_offset = 1024$ ， $m_base = data + 274$ ， $m_count = 300 - 274$ 。

写至 1#逻辑块底部，异步写回磁盘。

m_count 非 0，还要继续写。写操作导致文件长度增加， $i_size = 1024$ 。

2.3 写 2#逻辑块:

当前逻辑块 $bn = m_offset / 512 = 2$, 块内偏移量 0, 此次写 26 个字节。

混合索引表中 2#逻辑块的物理块号是 0, 系统为其分配新数据块 new, 登记:

$i_addr[2] = new$ 。

先读后写, 这是因为写入的字节数 26 不足 512 字节。系统为新物理块 new 分配缓存块, 启动 IO 操作、同步读入磁盘数据块 new。

iomove 将用户空间中的数据 $data[274] \sim data[299]$ 写入这块缓存。

修改 IO 参数, $m_offset = 1050$, $m_count = 0$ 。。。

未写至 1#逻辑块底部, 延迟写。缓存块打脏标记, 释放。

写操作导致文件长度增加, $i_size = 1050$ 。 m_count 是 0, write 系统调用结束。返回实际写入文件的字节数 300。

例题和习题3

一、完整的文件读写过程

2、识别文件的顺序读写操作 和 随机读写操作。每个系统调用完成后, $f_offset = \underline{\hspace{2cm}}$?

顺序读写

```
1、 fd = open ( ) ; // 0
   read (fd, ..., X) ; // X
   write (fd, ..., Y) ; // X+Y
   read (fd, ..., Z) ; // X+Y+Z
   .....
   close (fd) ;
```

随机读写

```
2、 fd = open ( ) ; // 0
   read (fd, ..., X) ; // X
   write (fd, ..., Y) ; // X+Y
   lseek (fd, SEEK_SET, 1000) ; // 1000
   read (fd, ..., Z) ; // 1000+Z
   .....
   close (fd) ;

3、 fd = creat( "newFile" , .....); // 0
   write (fd, ..., X) ; // X
   write (fd, ..., Y) ; // X+Y
   lseek (fd, SEEK_SET, 1000) ; // 1000
   read (fd, ..., Z) ; // 1000 + Z
   .....
   close (fd) ;
```

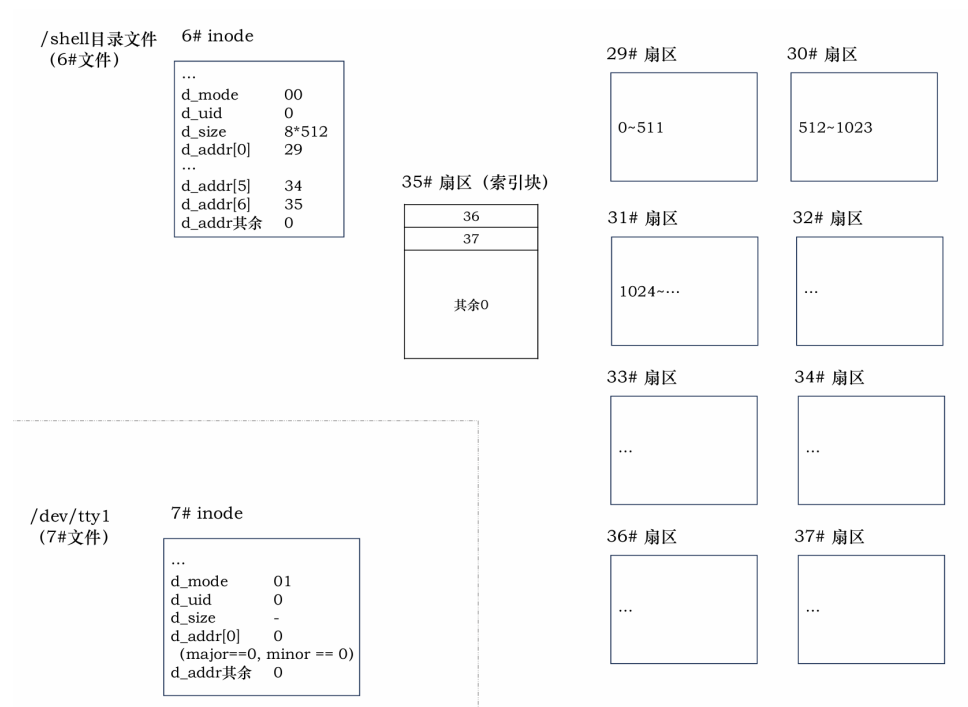
妈的每次读写都他妈在最后

若 P2 进程向文件追加写入 10000 个字符后关闭该文件, 引发 (A) 操作; 稍后, P1 关闭 example 文件, 引发 (ABC) 操作。

- A. 释放 file 结构
- B. 释放内存 i 节点
- C. 将内存 i 节点写回磁盘
- D. 不执行任何操作

盲猜是所有打开的关闭后才能写回释放内存, 没找到在哪, ppt太乱

E08：（即）文件系统 例题和习题1



d_size决定了有几个存数据的扇区，扇区个数决定了d_addr的个数，d_addr决定了扇区号
是否说明目录项一定都是从d_addr[0]所在的扇区开始存？？？

进程通信

进程通信1，第一部分并发不用看

- {
- 1.生产者/消费者
- 2.前5区/? ? 关系
- 3.读者/写者
- 4.理发师： PPT的第6页修正为上面这个版本（12/29日发的）
- 5.银行家算法，资源释放算法不要求写出来，（12/28日 ppt错了 银行家算法是死锁避免算法）
- 信号barbers初值是N，补全下面的代码那一题：这个题不对。不写了
- }

Part5 进程通信1 基础

互斥、同步、信号量

互斥:

- 互斥：一次只允许一个进程使用。
- 需要互斥保护的叫做临界资源，包括 共享变量（数据结构） 和 计算机系统中所有的外部设备。
- 临界区是访问共享变量的那段代码。

```
volatile int counter = 0;
int loops;

void *worker(void *arg)
{
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&m);
        counter++;
        Pthread_mutex_unlock(&m);
    }
    pthread_exit(NULL);
}
```

软件实现的锁:

```
worker () { // P1
    while ( )
    {
        L1: while( lock==1 )
            ;
        L2: lock = 1;

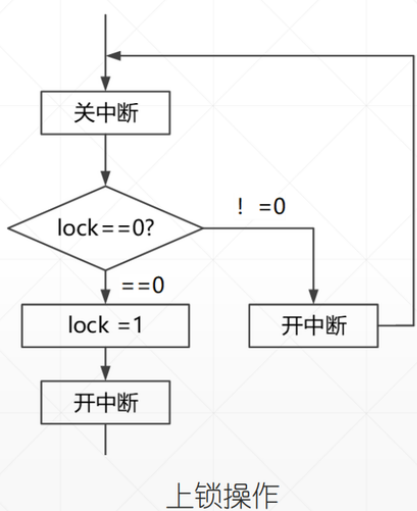
        counter++;

        lock = 0;
    }
}
```

正确的锁机制必须确保上锁、解锁操作的原子性，锁的检测和设置必需一次完成。

软件实现的锁可能被打断，产生错误。

硬件实现的锁:



- 锁的检测和设置是原子操作，执行期间进程不会放弃CPU。

- 缺点:
- 关中断上锁，在多处理器系统中是无效的。无法阻止其它CPU访问同一个变量。
 - 关中断时间必需足够短。这是因为，关中断时段，所有硬件瘫痪，系统无法与外界交互，不能响应紧急任务。所以，只适合用它来实现原子操作。
 - CLI, STI是操作系统的特权指令。APP无权使用。

信号量：

```
struct semaphore
{
    int value; // 值, 可供使用的资源数量
    struct sem_queue *head; // FIFO阻塞队列
                                等待使用资源的进程
} semaphore ;
```

value>0: R还有多少个资源可供使用。
value=0: R资源没了。阻塞队列为空。
value<0: value的绝对值表示阻塞队列中进程的数量。

访问共享资源前执行P操作

信号量的P操作
void p(S)
{
 S.value -= 1;
 if(S.value < 0)
 sleep();
}

检测、上锁、入队

共享资源使用完毕执行V操作

信号量的V操作
void v(S)
{
 S.value += 1;
 if(S.value <= 0)
 wakeup();
}

解锁、修正系统状态,
唤醒等待使用共享资源的进程

- 1、为共享资源 R 配一个信号量，用来描述资源使用状态。
- 2、进程访问共享资源前，**执行P操作**，同步等待资源访问权限。
- 3、访问结束后，**执行V操作**，释放资源访问权限、唤醒等待使用资源的另一个进程。

互斥锁：

semaphore mutex ; //保护共享变量 count

mutex.value = 1 ;

同步锁P：

观察系统状态并修正，维护系统状态一致性，这种操作叫做同步。实施同步的位置，叫做同步点。

Part5 进程通信2

生产者消费者

```
void producer()
{
    int item;
    while(true)
    {
        item = produceltem();
        p(empty);
        p(mutex);
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex);
        v(full);
    }
}

void consumer()
{
    int item;
    while(true)
    {
        p(full);
        p(mutex);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex);
        v(empty);
        consumeltem(item);
    }
}
```

同步信号量
full (初值为0)，数据的数量
empty (初值为N)，空缓存的数量

互斥信号量
mutex (初值为1)，缓存池的访问权限

读者/写者

写者操作（writer()函数）：

```
c

void writer() {
    while(true) {
        p(wmutex);    // 请求写者互斥信号量，获取访问资源的权限
        WRITEUNIT();  // 写者执行写操作
        v(wmutex);    // 释放写者互斥信号量，允许其他写者或读者操作资源
    }
}
```

读者操作（reader()函数）：

```
c


复制



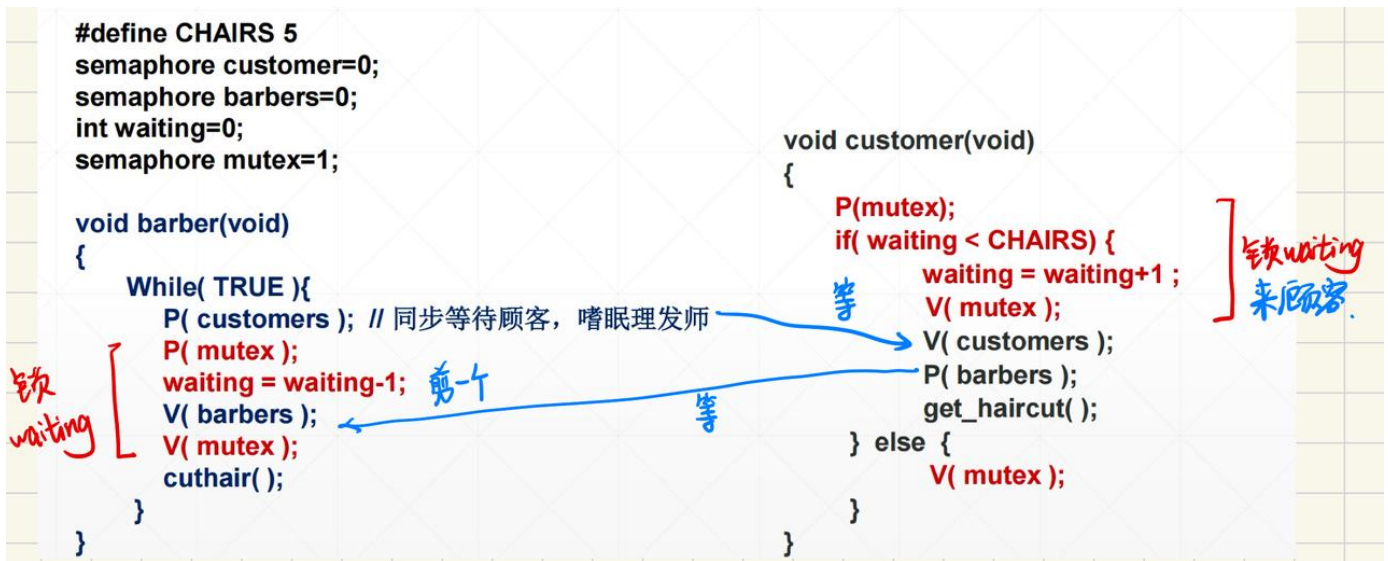
void reader() {
    while(true) {
        p(r);          // 请求读者信号量，表示有一个读者请求访问资源
        readcount++;    // 增加并发读者计数
        if (readcount == 1) {
            p(wmutex);  // 如果是第一个读者，锁住写者互斥信号量，阻止写者访问资源
        }
        v(r);          // 释放读者信号量，允许其他读者继续访问资源

        READUNIT();    // 读者执行读操作

        p(r);          // 请求读者信号量，表示读者操作结束
        readcount--;    // 减少并发读者计数
        if (readcount == 0) {
            v(wmutex);  // 如果是最后一个读者，释放写者互斥信号量，允许写者访问资源
        }
        v(r);          // 释放读者信号量，允许其他读者继续
    }
}
```

Part5 进程通信3

嗜眠理发师



哲学家进餐

死锁

- 互斥条件: 资源不可共享（软件：锁。 硬件：所有外设）
 - 不可剥夺条件: 指进程已获得的资源，在未使用完之前，不能被剥夺，只能使用完自己释放
 - 请求保持条件: 进程先申请一部分资源，再申请另一部分资源。在无法获得新的资源的时候，不放弃已有资源
 - 循环等待条件: 资源分配图里有环路
- 资源的固有属性。不可破坏。
- 破坏其中之一，就可以避免死锁
- 预防死锁 编码时破坏死锁产生的必要条件
 - 一次性给进程分配所需的全部资源 破坏请求保持条件
 - 保证资源分配图不出现环路 破坏循环等待条件
 - 避免死锁 运行时实施。评估进程提出的资源使用申请，避免系统陷入不安全状态。
 - 检测死锁 & 解除死锁
 - 为了提高系统的效率，不防范死锁（鸵鸟政策）
 - 定期执行检测工具，检测资源分配环路 和 陷入死锁的进程。
 - 逐个杀死重启进程，直至恢复正常运行状态。

解决死锁方法：

- 破坏请求保持条件：使用同步信号量，进程一次性申请和释放运行所需的全部资源。

2. 破坏循环等待条件：为资源编号，强制要求进程只有在获得编号小的资源之后才可以获得编号大的资源。

银行家算法：死锁避免策略

E09-1

E09-2 24版

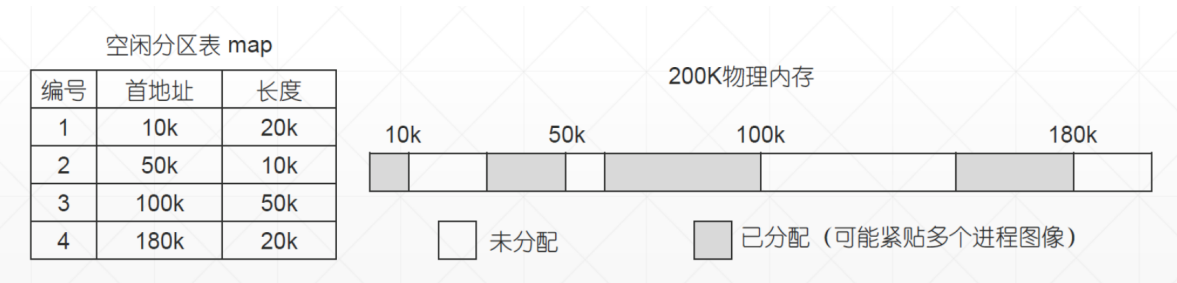
E09-3

存储管理

死锁PPT
进程管理7 exec和堆空间管理：习题里，exec的题目，期末考试会考

Part6 存储器管理1

用空闲分区表（map数组）登记所有空闲分区的首地址和长度。



重定位：将可执行程序中，指令和数据的逻辑地址转换成物理地址。

可执行程序中，指令和数据的地址是逻辑地址。

静态重定位由软件实施：程序加载时，根据进程图像分配得到的物理地址将所有逻辑地址转换为物理地址，修改代码段，用物理地址取代逻辑地址

动态重定位由CPU硬件，每次访存时实施：MMU将逻辑地址转换成物理地址

离散的存储管理方式：只看页式

a.地址映射和内存保护（段和页）

地址映射和内存保护

逻辑地址：段号段内偏移量 (offset)

段表寄存器

起始地址 (物理)长度 (6)

段表

base	limit	prot	段号
0x300	0x0b0	U, RO, X	0
0x400	0x150	U, RW, ~X	1
0x600	0x050	U, RW, ~X	2
0x800	0x020	U, RO, ~X	3
0xa00	0x1000	U, RW, ~X	4
0x2000	0x3000	U, RW, ~X	5

1、CPU内部有一个段表寄存器，引用现运行进程的段表

2、CPU给出的逻辑地址是2维的，包括段号和段内偏移量

3、MMU 地址映射和内存保护逻辑如下：

段号 >= 段表寄存器中的长度，异常

用段号做下标查段表，得到一个段表项

offset > limit，异常

写操作，prot是RO，异常

~X 取值，异常

物理地址 = base + offset

内存保护

地址映射

地址映射和内存保护

页表寄存器

起始地址F(物理)长度 (M)

页表

页号	base	prot
0	0x000	U RO P
1	0x000	U RW P
2	0x000	U RW P
...	...	...
M-1	0x000	U RW P

将逻辑地址分割成 页号P 和 页内偏移量offset

P >= M，异常（缺页中断）

用P做下标查页表，得到一个页表项pte

页表项的物理地址 = F + P * 页表项长度

pte为null，异常（缺页中断）

写操作，prot是RO，异常（缺页中断）

CPU用户态运行，prot是S，异常（缺页中断）

物理地址 = base * L + offset （L是页面的长度）

实现上，用base拼接offset

逻辑地址

物理地址

b.EAT

计算：EAT(Effective Access Time, 内存有效访问时间)

EAT = 访问页表时间 + 访问指令/数据时间

= TLB访问时间 + TLB不命中的概率 * 主存访问页表耗时 + 缓存访问时间 + 缓存不命中的概率 * 主存访问耗时

练习3：一级页表。

某一级页表内存管理系统，TLB 命中率是99%。从TLB中取PTE的平均耗时是10ns。指令、数据缓存命中率是95%。从缓存中取指令/数据的平均耗时是10ns。访问主存单元平均耗时200ns (取PTE、访问指令/数据相同)。求该系统内存有效访问时间。

EAT = (10+200*0.01) + (10+200*0.05) = 32ns

不设cache，EAT = 400ns。使用 cache，访存提速10倍以上。

二级页表。PTE没有命中TLB的话，MMU就。。。先去主存找PDE，然后去主存找PTE并缓存，最后去主存找数据

c.优点，缺点

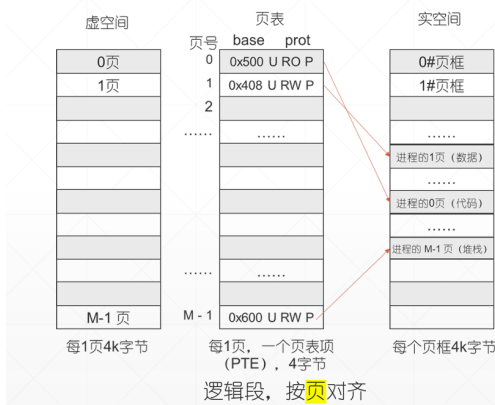
3、对连续存储管理方式的评论

- **优点：**管理起来方便。PCB中只需要登记进程图像在物理内存中的起始地址和长度
- **缺点：**
 - 存在碎片问题
 - 进程图像扩展开销巨大
 - 没办法利用程序运行局部性原理降低内存管理开销
- **克服缺点的方法：**
 - **离散存储管理方式。**解决碎片问题，提高物理内存资源利用率；还能够降低进程图像扩展开销。
 - **虚拟存储技术，**更加合理调配内存和盘交换区资源。让有限内存可以容纳更多SLOAD进程，提升系统整体性能。具体思路如下（1）盘交换区和文件系录，保存所有逻辑页。（2）内存存入存放近期需要访问的逻辑页。（3）请求调页。其余页面，访问时，缺页异常处理程序为其分配内存，从磁盘调入。

4、关于 Unix V6的讨论

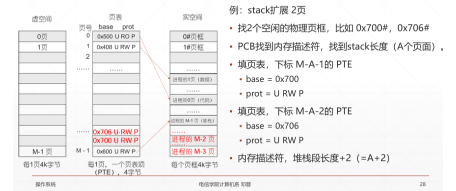
- Unix V6, Unix V6++是连续存储管理方式
 - 为正文段分配连续物理内存
 - 为可交换部分分配物理内存
 - 正文段与可交换部分不相邻，以利执行同一个应用程序的多个进程共享正文段
- 借助页表，将虚空间中不相邻的逻辑块映射至物理空间相邻的位置。这是为了避免为不存在的虚空间分配物理内存。
- 继承连续存储管理方式的所有优缺点。

页式存储管理的优点 1

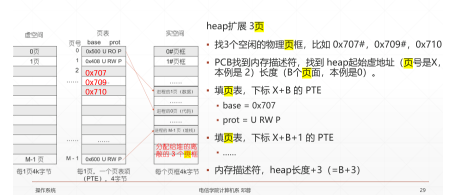


- **页式**，最显著的优点，**提高物理内存的利用率**。
- **原因**：每一个物理**页**框可以用来存放任何进程的任意逻辑**页**面。不存在外部碎片问题。
- **需要注意**：**页**内碎片是有的，每个逻辑段的末尾**页**，平均浪费半个**页**框的存储空间（2K字节）。

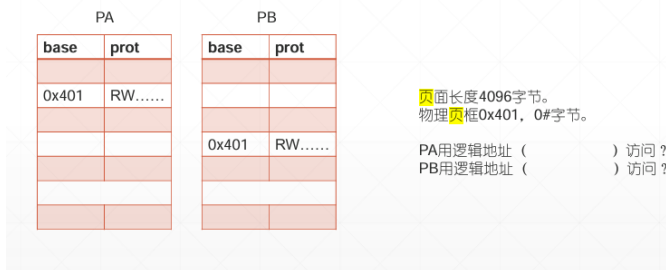
扩展进程图像极其容易，已有进程图像不必移动



扩展进程图像极其容易 (续)



页式存储管理的优点 3，以页为单位共享和保护物理内存



总结：分页机制存在2个主要问题

- 访存速度降低。因为每次访存前要多一次访存，访问页表生成物理地址。
- 每个进程一个页表。页表需要消耗大量连续物理内存。

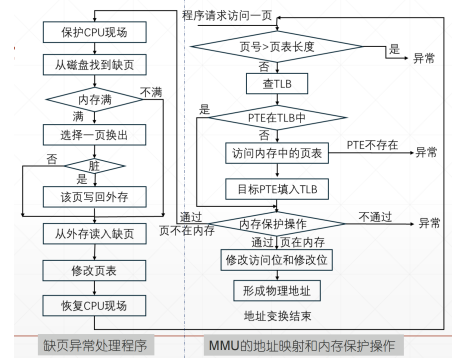
解决问题的方法

- **页表**需要消耗大量连续物理内存。二级、多级**页表**。
 - 若大片虚空间是空，无需为其设置**页表**。PTE不存啦。
 - 结合虚拟存储，请求调入**页表**，为目前需要使用的**页表**分配物理**页框**，其余**页表**留存盘交换区、甚至延迟创建。
- 访存速度降低。CPU加TLB单元。

Part6 存储器管理2 虚拟存储器

地址变换

- 1、逻辑地址取页号 (page)。
- 2、用页号查TLB。若不命中，从主存中的页表取PTE，填入TLB。
- 3、使用目标PTE，执行内存保护和请求调页操作。
 - 内存保护。权限检测未通过，抛出14#缺页异常、地址映射失败。
 - 请求调页。用存在位判断逻辑页是否驻留主存。
 - 若P==0，抛出14#缺页异常、请求调页。此次地址映射结束。
 - 若P==1，地址映射（生成物理地址），
PTE维护（修改 prot：访问位置1、修改位置1。
TLB内存同步修改）。



4、例题

	主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
						
0x266	0x5123	0x200	1	0	1	U RO
0x267	0x3567	0x420	无意义	无意义	0	U RW

将线性地址翻译成物理地址。写出详细的地址映射过程。

- 1、线性地址0x266000，读操作
- 2、线性地址0x266000，写操作
- 3、线性地址0x267100，写操作

主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
.....					
0x266	0x5123	0x200	1	0	1 U RO
0x267	0x3567	0x420	无意义	无意义	0 U RW

- 1、线性地址0x266000，读操作
• 页号：0x266。PTE存在且P bit是1，主存命中。
• 内存保护操作：
• RO，读操作合法
• PTE.access = 1;
• 生成物理地址：0x5123000
- 2、线性地址0x266000，写操作
• 页号：0x266。PTE存在且P bit是1，主存命中。
• 内存保护操作：
• RO，写操作非法。抛出14#缺页异常。

14#异常处理程序：prot不对，这是非法内存访问，向现运行进程发信号SIGSEGV，杀死出错进程，阻止应用程序的非法内存访问。

操作系统

电信学院计算机系 邓晋

16

- 3、线性地址0x267100，写操作

• 页号：0x267。PTE存在。

• 内存保护操作。

• PTE, RW bit是1。写操作合法

• 请求调页操作

• PTE, Present bit是0，抛出缺页异常，引发缺页的线性地址0x267100 → cr2

• 执行缺页异常处理程序

• 为0x267#逻辑页面（cr2去掉末尾100）分配一个物理页框，0x5555。

• 假设0x5555不脏，修改映射至0x5555页框的PTE，P=0。

• 调页：向磁盘发IO命令后，进程入睡放弃CPU。IO参数如下：

• 内存首地址：0x5555000

• 磁盘起始扇区：0x420（0x267#逻辑页面在磁盘上的首地址）

• 数据传输量：4096字节

• 读操作（扇区0x420，0x421，0x422，...，0x427 → 0x5555页框）

• 修改0x267#PTE：主存页框号=0x5555，P=1，修改位=0，访问字段=0

• 缺页异常返回，重新执行访问线性地址0x267100的指令

• 页号：0x267，PTE present bit为1，进程拥有本页面的写权限。

• 修改位=1，访问字段=1

• 生成物理地址：0x5555100

主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
.....					
0x266	0x5123	0x200	1	0	1 U RW
0x267	0x3567	0x420	无意义	无意义	0 U RW

IO完成，磁盘中断处理程序唤醒本进程

异常处理程序继续执行

主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
.....					
0x266	0x5123	0x200	1	0	1 U RO
0x267	0x3567	0x420	1	1	1 U RW

操作系统

电信学院计算机系 邓晋

17

- 2、线性地址0x267100，写操作

• 页号：0x267。PTE存在。

• 内存保护操作。

• PTE, RW bit是1。写操作合法

• 请求调页操作

• PTE, Present bit是0，抛出缺页异常，线性地址0x267100 → cr2

• 执行缺页异常处理程序

• 为0x267#逻辑页面（cr2去掉末尾100）分配一个物理页框，0x5555。

• 假设0x5555脏。

主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
.....					
0x266	0x5123	0x200	1	0	1 U RW
0x267	0x3567	0x420	无意义	无意义	0 U RW

0x5555#物理页框写回磁盘交换区的过程：

- 1、若PTE外存地址为空，磁盘交换区分配4096字节（连续的8个扇区），外存地址=起始扇区号start。
- 2、0x5555#物理页框写回磁盘，从start开始，连续写8个扇区。
- 3、修改映射至0x5555页框的PTE：P=0。

• 调页：向磁盘发IO命令后，进程入睡放弃CPU。IO参数如下：

• 内存首地址：0x5555000

• 磁盘起始扇区：0x420（0x267#逻辑页面在磁盘上的首地址）

• 数据传输量：4096字节

• 读操作（扇区0x420，0x421，0x422，...，0x427 → 0x5555页框）

• 修改0x267#PTE：主存页框号=0x5555，P=1，修改位=0，访问字段=0

• 缺页异常返回，重新执行访问线性地址0x267100的指令

• 页号：0x267，PTE present bit为1，进程拥有本页面的写权限。

• 修改位=1，访问字段=1

• 生成物理地址：0x5555100

主存页框号	外存地址	访问字段 (access)	修改位 (dirty)	状态位 (present)	U/S R/W & others
.....					
0x266	0x5123	0x200	1	0	1 U RW
0x267	0x3567	0x420	1	1	1 U RW

操作系统

电信学院计算机系 邓晋

18

（cr2 去掉末尾100）分配一个物理页框，0x5555。（0x5555是随便找的一个，为什么不放在0x3567可能是规定的。。。）

置换算法的实现

5、页面置换算法（使用固定分配局部置换的页面替换策略）

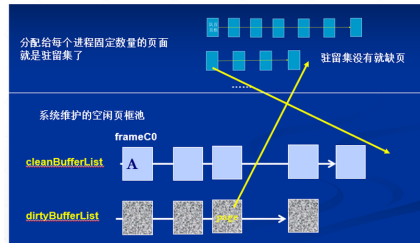
缺页时，若内存已无空闲空间，淘汰一页。用新逻辑页面覆盖被淘汰页面占用的物理页框。若被淘汰页面脏（修改过），写回磁盘。

页面置换算法决定淘汰哪一页。优化目标：降低缺页率。

- 1、OPT，未来访问时刻最远的（最优）
- 2、FIFO，最先进入内存的
- 3、LRU，最近最久未访问的（Least Recently Used）
- 4、LRU近似算法，比如clock
- 5、随机，随便

FIFO&页面缓存算法

2、页面缓存算法（空闲页框池里有所缺页面）



每个进程，一条FIFO队列。进程访问的逻辑页面FIFO队列不存在，缺页。

系统维护空闲页框池。其中的物理页框所有进程共用。
空闲页框池有两条LRU队列：干净缓存队列和脏缓存队列。

缺页时:

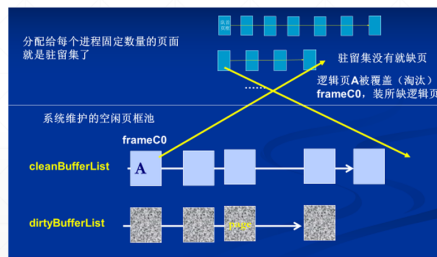
- 1、看下空闲页框池里有没有？
Y，摘出来，挂进FIFO队尾。
N，用frameC0表补缺页面，原先里面的逻辑页A被淘汰（里面装的是谁的页面？不知道，不要紧）。frameC0挂队尾
- 2、摘FIFO队头为首，挂空闲页框池的队尾。

操作系统

电信学院计算机系 邓蓉

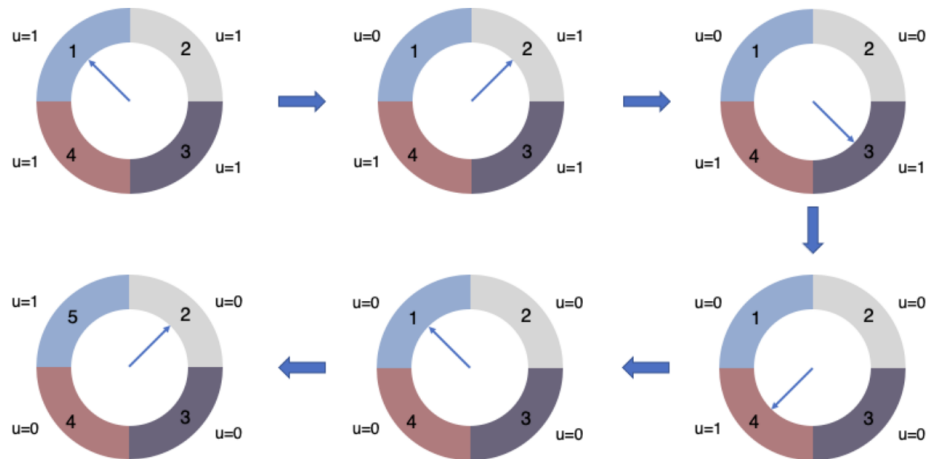
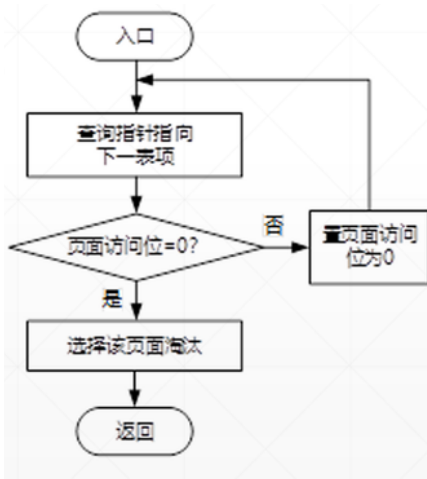
35

页面缓存算法（空闲页框池里没有所缺页面）



- 1、看下空闲页框池里有没有？
Y，摘出来，挂队尾。
N，用frameC0装所缺页面，原先里面的逻辑页A被淘汰（里面装的是谁的页面？不知道，不要紧）。
frameC0挂FIFO队列队尾
2、摘FIFO队列队首，挂空闲页框池的队尾。

Clock算法



改进型Clock置换算法 —— 优先淘汰干净的页面

访问位A和修改位(脏位)D的4种组合。描述了页框的四种不同的类型。

- 1类: $A=0, D=0$ 最近没有被访问, 也没有被修改;
- 2类: $A=0, D=1$ 最近没有被访问, 但被修改;
- 3类: $A=1, D=0$ 最近被访问过, 但没有被修改;
- 4类: $A=1, D=1$ 最近被访问过, 被修改;

淘汰这四种页面，代价？

改进型Clock置换算法的实现

显然第1类页面是最佳淘汰页面，应尽量避免淘汰第4类页面。算法执行的步骤如下：

- 1 从current指针开始扫描内存页框，寻找第1类页面（未访问过的干净页面）予以替换。
- 2 如果第1步操作失败，重新扫描，寻找第2类页面（未访问过的脏页面），对扫描过程中途径的所有页帧A位清0。
- 3 如果第2步也失败，指针将回到原来的位置（所有页帧A位为0）。重复第1步，必要时重复第2步。一定能找到淘汰页（最近被访问过的页面）。

工作集/驻留集

请求调页虚拟存储器相关的理论——工作集和驻留集

- **进程 P_i 的工作集：**未来一小段时间内，需要使用的所有逻辑页面。工作集大小 N_i 。
- **进程的驻留集：**当前存储在内存中的所有逻辑页面
- **工作集是驻留集的子集，不会缺页**
- **当应用程序从一个计算阶段过渡到下个计算阶段时，工作集会发生变化。这个阶段进程会缺页，持续一段时间。之后，进程稳定工作于下个计算阶段，不再会经常缺页。**
- **如果一个进程频繁缺页，就该为它追加主存页框。**

抖动

请求调页虚拟存储器相关的理论——抖动

- 如果系统接纳了过多的作业，就会出现主存页框不足，无法容纳所有进程工作集的情况，此时所有进程均缺页频繁。
- $\sum N_i \gg$ 内存总容量，系统抖动。主机和主硬盘忙于缺页异常任务，无法执行有效计算。
- 抖动的解决方法：将部分进程挂起，驻留集全部交换至磁盘，空出来的物理页框分配给留在内存的其它进程。保证内存中的进程可以顺利运行。

E10 思考题不看

exec